

Data Visualization and Statistics in R

Aakash Yadav

Contents

Cover	5
Preface	7
Download the Book	9
Acknowledgments	11
About the Author	13
1 Introduction	15
1.1 The ecosystem	15
1.2 Data Types	16
2 Data Types	25
2.1 Vectors	25
2.2 Checking data type	25
2.3 Coercion	26
2.4 Lists	26
2.5 Matrices	26
2.6 Arrays	26
2.7 Factors	27
2.8 Data Frames	27
2.9 Names	28
2.10 Subsetting	28
2.11 Installing and loading packages	29
2.12 Getting Help	29
2.13 Looking at Data	29
3 Making your First Plot	35
3.1 Loading the data	35
3.2 Plot the Data Layer	36
3.3 Plot the Aesthetic Layer	36
3.4 Plot the Geometric Layer	37

3.5	Plot the Statistical Layer	38
3.6	Plot the Coordinate Layer	39
3.7	Plot the Facet Layer	40
3.8	Plot the Scale Layer	41
3.9	Plot the Label Layer	42
3.10	Plot the Theme Layer	43
4	Additional Customizations	45
4.1	Customizing Aesthetics	45
4.2	Combining Multiple Plots	46
4.3	Interactive Plots	47
4.4	Color and Fill Mappings	48
4.5	Advanced Theme Customization	51
4.6	Legend Customization	53
4.7	Annotations	56
4.8	Axis Customization	59
4.9	Different Plot Types	62
4.10	Statistical Transformations	73
4.11	Faceting Options	76
4.12	Handling Overplotting	79
4.13	Error Bars and Confidence Intervals	82
4.14	Working with Multiple Datasets	84
4.15	Best Practices	85
4.16	Saving Plots	88
5	Statistical Analysis	89
5.1	Descriptive Statistics	89
5.2	t-Test	90
5.3	Correlation	91

Cover

Preface

Welcome to the world of R—a powerful and versatile programming language that opens the doors to data exploration, statistical analysis, and stunning data visualizations. Whether you’re a complete beginner or an experienced programmer seeking to expand your skills, this book is your comprehensive guide to mastering R from scratch. The book also draws from AI generated content, however, the content is heavily edited and curated by the author to ensure the highest quality.

Foundations of R

We embark on a journey through the fundamental principles of the R language. From understanding basic data types to mastering control structures and functions, each chapter is crafted to provide a solid foundation for your programming endeavors. Step by step, we unravel the mysteries of R syntax, empowering you to write efficient and elegant code.

Unleashing the Power of Visualization

With a strong grasp of R basics, we dive headfirst into the world of expert-level graphing. Visualization is not just about creating pretty pictures; it’s about telling a compelling story with your data. Learn to wield the powerful `ggplot2` package with finesse, producing a wide array of static and interactive visualizations that breathe life into your data.

Demystifying Statistics

Statistics can be intimidating, but fear not! In the third section, we unravel the complexities of statistical analysis using R. From descriptive statistics to hypothesis testing and regression analysis, each concept is demystified with clear explanations and hands-on examples. Gain the confidence to make data-driven decisions and draw meaningful insights from your analyses.

Who Should Read This Book?

Whether you’re a student, a data enthusiast, or a researcher, if you’re intrigued by the possibilities that data holds, this book is for you. It is good to have some prior programming experience, but it is not required. If you’re new to programming, you’ll find the book to be a gentle introduction to the world

of coding. If you're an experienced programmer, you'll find the book to be a comprehensive guide to data visualization and statistical analysis using R.

Happy coding!

Aakash Yadav

Download the Book

You can download the pdf version of the book from [here](#).

Acknowledgments

This (incomplete) book is heavily inspired by the teaching of the Interactive Data Visualization course taught by Dr. Anthony McDonald at the University of Wisconsin-Madison.

This is an incomplete undergoing project, contribution in any form is highly appreciated, be it reporting issues, suggesting improvements, or creating pull requests on GitHub: <https://github.com/nimrobotics/r-for-research>.

About the Author

Aakash Yadav is a PhD student in the Department of Industrial and Systems Engineering at the University of Wisconsin-Madison. His research focuses on understanding trust in human-robot interaction. His research interests span human-robot interaction, disaster robotics, machine learning, human factors, cognition, and affective computing. I work towards understanding multi-human-robot collaboration to improve teaming, system performance, and safety. He strongly believes in the power of open science and open source. He is passionate about teaching and mentoring.

To learn more about his research, visit his website: nimrobotics.com.

Chapter 1

Introduction

R is a programming language and software environment for statistical computing and graphics. It is widely used among statisticians and data miners for developing statistical software and data analysis. It has a command line interface, but there are several graphical user interfaces that are also available for use with R.

- R is a free and open-source software.
- It provides a wide variety of statistical and graphical techniques.
- R has a large and active community of users and developers.
- It has a powerful and flexible programming language.
- R can be extended through packages.

1.1 The ecosystem

1.1.1 R

R is a programming language and software environment for statistical computing and graphics. It is widely used among statisticians and data miners for developing statistical software and data analysis.

1.1.2 CRAN

The Comprehensive R Archive Network (CRAN) is a network of servers around the world that contain the source code, documentation, and add-on packages for R. The CRAN website is <https://cran.r-project.org/>. Most of the packages can be installed directly from CRAN using the `install.packages()` function.

1.1.3 RStudio

RStudio is an integrated development environment (IDE) for R. It includes a console, syntax-highlighting editor that supports direct code execution, as well

as tools for plotting, history, debugging and workspace management.

1.1.4 R Markdown

R Markdown is a file format for making dynamic documents with R. An R Markdown document is written in markdown (an easy-to-write plain text format) and contains chunks of embedded R code, like the document below.

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
1+3 # This is a comment
```

```
## [1] 4
```

1.2 Data Types

1.2.1 Vectors

Vectors are the most basic R data objects and there are six types of atomic vectors. They are logical, integer, double, character, complex and raw.

```
# Logical
x <- TRUE
# Integer
x <- 1L
# Double
x <- 1
# Character
x <- "a"
# Complex
x <- 1 + 0i
# Raw
x <- charToRaw("a")
```

Create a vector using the combine function `c()`. The elements of a vector all have the same type. `c` stands for combine.

```
x <- c(1, 2, 3, 4)
x
```

```
## [1] 1 2 3 4
```


1.2.2 Checking data type

```
x <- 1
typeof(x)
```

```
## [1] "double"
```

1.2.3 Coercion

R is a weakly typed language. This means that the data type of a variable can be changed by another assignment of a different data type to the same variable. This is called coercion.

```
x <- 1
typeof(x)
```

```
## [1] "double"
```

```
x <- "a"
typeof(x)
```

```
## [1] "character"
```

1.2.4 Lists

Lists are a special type of vector that can contain elements of different classes. Lists are a very important data type in R and are used to represent data frames, among other things.

```
x <- list(1, "a", TRUE, 1 + 0i)
```

1.2.5 Matrices

Matrices are vectors with a dimension attribute. The dimension attribute is itself an integer vector of length 2 (nrow, ncol).

```
m <- matrix(nrow = 2, ncol = 3)
m
```

```
##      [,1] [,2] [,3]
## [1,]  NA  NA  NA
## [2,]  NA  NA  NA
dim(m)
```

```
## [1] 2 3
```

1.2.6 Arrays

Arrays are the general form of matrices, where the dimension attribute is an integer vector of any length.

```
a <- array(1:6, c(2, 3))
a
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
dim(a)
```

```
## [1] 2 3
```

1.2.7 Factors

Factors are used to represent categorical data and can be unordered or ordered. Factors can be viewed as integer vectors where each integer has a label.

```
x <- factor(c("yes", "yes", "no", "yes", "no"))
x
```

```
## [1] yes yes no  yes no
## Levels: no yes
```

```
table(x)      # Frequency table
```

```
## x
##  no yes
##   2   3
```

```
unclass(x)    # Integer representation
```

```
## [1] 2 2 1 2 1
## attr("levels")
## [1] "no"  "yes"
```

```
attr(x, "levels") # Levels of factor
```

```
## [1] "no"  "yes"
```

```
attr(x, "class")  # Class of factor
```

```
## [1] "factor"
```

1.2.8 Data Frames

Data frames are used to store tabular data. They are represented as a special type of list where every element of the list has to have the same length.

```
x <- data.frame(foo = 1:4, bar = c(T, T, F, F))
x
```

```
##   foo  bar
## 1   1 TRUE
## 2   2 TRUE
## 3   3 FALSE
## 4   4 FALSE
```

```
nrow(x)
```

```
## [1] 4
```

```
ncol(x)
```

```
## [1] 2
```

1.2.9 Names

All objects in R have a name attribute. The name attribute is used to attach additional information about the object. It is most often used to give names to the elements of a list, vectors and data frames.

```
x <- 1:3
```

```
names(x)
```

```
## NULL
```

```
names(x) <- c("foo", "bar", "norf")
```

```
x
```

```
##   foo  bar norf
##    1    2    3
```

1.2.10 Subsetting

Subsetting is the process of extracting subsets of objects. Subsetting is most commonly performed on vectors, matrices and data frames.

```
x <- c("a", "b", "c", "c", "d", "a")
x[1] # First element
```

```
## [1] "a"
```

```
x[2] # Second element
```

```
## [1] "b"
```

```
x[1:3] # First three elements
```

```
## [1] "a" "b" "c"
```

```
x[x > "a"] # Elements greater than "a"
```

```
## [1] "b" "c" "c" "d"
```

1.2.11 Installing and loading packages

```
# install.packages("ggplot2") # install package. Only need to do this once
library(ggplot2)             # load package. Need to do this every time you start R
```

1.2.12 Getting Help

```
help(plot)    # Help on a specific topic
?plot         # Help on a specific topic
```

1.2.13 Looking at Data

```
head(mtcars)    # First few rows of data
```

```
##           mpg cyl  disp  hp  drat    wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160 110  3.90  2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21.0   6  160 110  3.90  2.875 17.02  0  1    4    4
## Datsun 710      22.8   4  108  93  3.85  2.320 18.61  1  1    4    1
## Hornet 4 Drive  21.4   6  258 110  3.08  3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175  3.15  3.440 17.02  0  0    3    2
## Valiant         18.1   6  225 105  2.76  3.460 20.22  1  0    3    1
```

```
tail(mtcars)    # Last few rows of data
```

```
##           mpg cyl  disp  hp  drat    wt  qsec vs am gear carb
## Porsche 914-2  26.0   4 120.3  91  4.43  2.140 16.7   0  1    5    2
## Lotus Europa   30.4   4  95.1 113  3.77  1.513 16.9   1  1    5    2
## Ford Pantera L 15.8   8 351.0 264  4.22  3.170 14.5   0  1    5    4
## Ferrari Dino   19.7   6 145.0 175  3.62  2.770 15.5   0  1    5    6
## Maserati Bora   15.0   8 301.0 335  3.54  3.570 14.6   0  1    5    8
## Volvo 142E      21.4   4 121.0 109  4.11  2.780 18.6   1  1    4    2
```

```
str(mtcars)     # Structure of data
```

```
## 'data.frame':    32 obs. of  11 variables:
## $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
## $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
## $ disp: num  160 160 108 258 360 ...
## $ hp : num  110 110 93 110 175 105 245 62 95 123 ...
## $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
## $ wt : num  2.62 2.88 2.32 3.21 3.44 ...
```

```
## $ qsec: num 16.5 17 18.6 19.4 17 ...
## $ vs : num 0 0 1 1 0 1 0 1 1 1 ...
## $ am : num 1 1 1 0 0 0 0 0 0 0 ...
## $ gear: num 4 4 4 3 3 3 3 4 4 4 ...
## $ carb: num 4 4 1 1 2 1 4 2 2 4 ...
```

```
summary(mtcars) # Summary of data
```

```
##      mpg          cyl          disp          hp          drat          wt
##  Min.   :10.40   Min.   :4.000   Min.   : 71.1   Min.   : 52.0   Min.   :2.760   Min.   :1.513
## 1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5   1st Qu.:3.080   1st Qu.:2.581
## Median :19.20   Median :6.000   Median :196.3   Median :123.0   Median :3.695   Median :3.325
## Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7   Mean   :3.597   Mean   :3.217
## 3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0   3rd Qu.:3.920   3rd Qu.:3.610
## Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0   Max.   :4.930   Max.   :5.424
##      vs          am          gear          carb
##  Min.   :0.0000   Min.   :0.0000   Min.   :3.000   Min.   :1.000
## 1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000
## Median :0.0000   Median :0.0000   Median :4.000   Median :2.000
## Mean   :0.4375   Mean   :0.4062   Mean   :3.688   Mean   :2.812
## 3rd Qu.:1.0000   3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000
## Max.   :1.0000   Max.   :1.0000   Max.   :5.000   Max.   :8.000
```

You can also use the `View()` function to view the data in a spreadsheet like format.

```
View(mtcars)
```

Or you can use the `edit()` function to edit the data in a spreadsheet like format.

```
#edit(mtcars)
```

1.2.13.1 Names of the columns

```
names(mtcars)
```

```
## [1] "mpg" "cyl" "disp" "hp" "drat" "wt" "qsec" "vs" "am" "gear" "carb"
```

1.2.13.2 Number of rows and columns

```
dim(mtcars) # Number of rows and columns
```

```
## [1] 32 11
```

```
nrow(mtcars) # Number of rows
```

```
## [1] 32
```

```
ncol(mtcars) # Number of columns
```

```
## [1] 11
```

1.2.13.3 Accessing the data

```
mtcars[1, 1] # First row, first column
```

```
## [1] 21
```

```
mtcars[1, ] # First row, all columns
```

```
##           mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4  21   6  160 110  3.9 2.62 16.46  0  1    4    4
```

```
mtcars[, 1] # All rows, first column
```

```
## [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4
## [23] 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7 15.0 21.4
```

```
mtcars[1:5, ] # First five rows, all columns
```

```
##           mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160 110  3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21.0   6  160 110  3.90 2.875 17.02  0  1    4    4
## Datsun 710      22.8   4  108  93  3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive  21.4   6  258 110  3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175  3.15 3.440 17.02  0  0    3    2
```

```
mtcars[, 1:5] # All rows, first five columns
```

```
##           mpg cyl  disp  hp drat
## Mazda RX4      21.0   6 160.0 110 3.90
## Mazda RX4 Wag  21.0   6 160.0 110 3.90
## Datsun 710      22.8   4 108.0  93 3.85
## Hornet 4 Drive  21.4   6 258.0 110 3.08
## Hornet Sportabout 18.7   8 360.0 175 3.15
## Valiant         18.1   6 225.0 105 2.76
## Duster 360      14.3   8 360.0 245 3.21
## Merc 240D       24.4   4 146.7  62 3.69
## Merc 230        22.8   4 140.8  95 3.92
## Merc 280        19.2   6 167.6 123 3.92
## Merc 280C       17.8   6 167.6 123 3.92
## Merc 450SE      16.4   8 275.8 180 3.07
## Merc 450SL      17.3   8 275.8 180 3.07
## Merc 450SLC     15.2   8 275.8 180 3.07
## Cadillac Fleetwood 10.4   8 472.0 205 2.93
## Lincoln Continental 10.4   8 460.0 215 3.00
```

```
## Chrysler Imperial    14.7    8 440.0 230 3.23
## Fiat 128              32.4    4  78.7  66 4.08
## Honda Civic           30.4    4  75.7  52 4.93
## Toyota Corolla        33.9    4  71.1  65 4.22
## Toyota Corona         21.5    4 120.1  97 3.70
## Dodge Challenger      15.5    8 318.0 150 2.76
## AMC Javelin           15.2    8 304.0 150 3.15
## Camaro Z28            13.3    8 350.0 245 3.73
## Pontiac Firebird      19.2    8 400.0 175 3.08
## Fiat X1-9             27.3    4  79.0  66 4.08
## Porsche 914-2         26.0    4 120.3  91 4.43
## Lotus Europa          30.4    4  95.1 113 3.77
## Ford Pantera L        15.8    8 351.0 264 4.22
## Ferrari Dino          19.7    6 145.0 175 3.62
## Maserati Bora         15.0    8 301.0 335 3.54
## Volvo 142E            21.4    4 121.0 109 4.11
```

```
mtcars[1:5, 1:5] # First five rows, first five columns
```

```
##              mpg cyl disp  hp drat
## Mazda RX4      21.0   6  160 110 3.90
## Mazda RX4 Wag  21.0   6  160 110 3.90
## Datsun 710     22.8   4  108  93 3.85
## Hornet 4 Drive  21.4   6  258 110 3.08
## Hornet Sportabout 18.7   8  360 175 3.15
```

```
mtcars[1:5, c(1, 3, 5)] # First five rows, columns 1, 3 and 5
```

```
##              mpg disp drat
## Mazda RX4      21.0  160 3.90
## Mazda RX4 Wag  21.0  160 3.90
## Datsun 710     22.8  108 3.85
## Hornet 4 Drive  21.4  258 3.08
## Hornet Sportabout 18.7  360 3.15
```

```
mtcars[1:5, c("mpg", "disp", "hp")] # First five rows, columns mpg, disp and hp
```

```
##              mpg disp  hp
## Mazda RX4      21.0  160 110
## Mazda RX4 Wag  21.0  160 110
## Datsun 710     22.8  108  93
## Hornet 4 Drive  21.4  258 110
## Hornet Sportabout 18.7  360 175
```

1.2.13.4 Accessing the data using column names

Use the `$` operator to access the data using column names.

```
mtcars$mpg # Accessing the mpg column

## [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4
## [23] 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7 15.0 21.4

mtcars$mpg[1] # Accessing the first element of the mpg column

## [1] 21

mtcars$mpg[1:5] # Accessing the first five elements of the mpg column

## [1] 21.0 21.0 22.8 21.4 18.7

mtcars$mpg[mtcars$mpg > 20] # Accessing the elements of the mpg column that are greater
                             than 20

## [1] 21.0 21.0 22.8 21.4 24.4 22.8 32.4 30.4 33.9 21.5 27.3 26.0 30.4 21.4
```


Chapter 2

Data Types

2.1 Vectors

Vectors are the most basic R data objects and there are six types of atomic vectors. They are logical, integer, double, character, complex and raw.

```
# Logical
x <- TRUE
# Integer
x <- 1L
# Double
x <- 1
# Character
x <- "a"
# Complex
x <- 1 + 0i
# Raw
x <- charToRaw("a")
```

Create a vector using the combine function `c()`. The elements of a vector all have the same type. `c` stands for combine.

```
x <- c(1, 2, 3, 4)
x
```

```
## [1] 1 2 3 4
```

2.2 Checking data type

```
x <- 1
typeof(x)
```

```
## [1] "double"
```

2.3 Coercion

R is a weakly typed language. This means that the data type of a variable can be changed by another assignment of a different data type to the same variable. This is called coercion.

```
x <- 1
typeof(x)
```

```
## [1] "double"
```

```
x <- "a"
typeof(x)
```

```
## [1] "character"
```

2.4 Lists

Lists are a special type of vector that can contain elements of different classes. Lists are a very important data type in R and are used to represent data frames, among other things.

```
x <- list(1, "a", TRUE, 1 + 0i)
```

2.5 Matrices

Matrices are vectors with a dimension attribute. The dimension attribute is itself an integer vector of length 2 (nrow, ncol).

```
m <- matrix(nrow = 2, ncol = 3)
m
```

```
##      [,1] [,2] [,3]
## [1,]  NA  NA  NA
## [2,]  NA  NA  NA
```

```
dim(m)
```

```
## [1] 2 3
```

2.6 Arrays

Arrays are the general form of matrices, where the dimension attribute is an integer vector of any length.

```
a <- array(1:6, c(2, 3))
a
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
dim(a)
```

```
## [1] 2 3
```

2.7 Factors

Factors are used to represent categorical data and can be unordered or ordered. Factors can be viewed as integer vectors where each integer has a label.

```
x <- factor(c("yes", "yes", "no", "yes", "no"))
x
```

```
## [1] yes yes no  yes no
## Levels: no yes
```

```
table(x)      # Frequency table
```

```
## x
##  no yes
##   2   3
```

```
unclass(x)    # Integer representation
```

```
## [1] 2 2 1 2 1
## attr("levels")
## [1] "no"  "yes"
```

```
attr(x, "levels") # Levels of factor
```

```
## [1] "no"  "yes"
```

```
attr(x, "class")  # Class of factor
```

```
## [1] "factor"
```

2.8 Data Frames

Data frames are used to store tabular data. They are represented as a special type of list where every element of the list has to have the same length.

```
x <- data.frame(foo = 1:4, bar = c(T, T, F, F))
x
```

```
##   foo   bar
## 1    1  TRUE
## 2    2  TRUE
## 3    3 FALSE
## 4    4 FALSE
```

```
nrow(x)
```

```
## [1] 4
```

```
ncol(x)
```

```
## [1] 2
```

2.9 Names

All objects in R have a name attribute. The name attribute is used to attach additional information about the object. It is most often used to give names to the elements of a list, vectors and data frames.

```
x <- 1:3
```

```
names(x)
```

```
## NULL
```

```
names(x) <- c("foo", "bar", "norf")
```

```
x
```

```
##   foo   bar norf
##    1    2    3
```

2.10 Subsetting

Subsetting is the process of extracting subsets of objects. Subsetting is most commonly performed on vectors, matrices and data frames.

```
x <- c("a", "b", "c", "c", "d", "a")
```

```
x[1]    # First element
```

```
## [1] "a"
```

```
x[2]    # Second element
```

```
## [1] "b"
```

```
x[1:3]  # First three elements
```

```
## [1] "a" "b" "c"
```

```
x[x > "a"] # Elements greater than "a"
```

```
## [1] "b" "c" "c" "d"
```

2.11 Installing and loading packages

```
# install.packages("ggplot2") # install package. Only need to do this once
library(ggplot2)              # load package. Need to do this every time you start R
```

2.12 Getting Help

```
help(plot)      # Help on a specific topic
?plot           # Help on a specific topic
```

2.13 Looking at Data

```
head(mtcars)     # First few rows of data
```

```
##           mpg cyl  disp  hp  drat    wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160 110  3.90  2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21.0   6  160 110  3.90  2.875 17.02  0  1    4    4
## Datsun 710      22.8   4  108  93  3.85  2.320 18.61  1  1    4    1
## Hornet 4 Drive  21.4   6  258 110  3.08  3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175  3.15  3.440 17.02  0  0    3    2
## Valiant         18.1   6  225 105  2.76  3.460 20.22  1  0    3    1
```

```
tail(mtcars)     # Last few rows of data
```

```
##           mpg cyl  disp  hp  drat    wt  qsec vs am gear carb
## Porsche 914-2  26.0   4 120.3  91  4.43  2.140 16.7   0  1    5    2
## Lotus Europa   30.4   4  95.1 113  3.77  1.513 16.9   1  1    5    2
## Ford Pantera L 15.8   8 351.0 264  4.22  3.170 14.5   0  1    5    4
## Ferrari Dino   19.7   6 145.0 175  3.62  2.770 15.5   0  1    5    6
## Maserati Bora   15.0   8 301.0 335  3.54  3.570 14.6   0  1    5    8
## Volvo 142E      21.4   4 121.0 109  4.11  2.780 18.6   1  1    4    2
```

```
str(mtcars)      # Structure of data
```

```
## 'data.frame':   32 obs. of  11 variables:
## $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
## $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
## $ disp: num  160 160 108 258 360 ...
## $ hp : num  110 110 93 110 175 105 245 62 95 123 ...
## $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
## $ wt : num  2.62 2.88 2.32 3.21 3.44 ...
## $ qsec: num  16.5 17 18.6 19.4 17 ...
```

```
## $ vs : num 0 0 1 1 0 1 0 1 1 1 ...
## $ am : num 1 1 1 0 0 0 0 0 0 0 ...
## $ gear: num 4 4 4 3 3 3 3 4 4 4 ...
## $ carb: num 4 4 1 1 2 1 4 2 2 4 ...
```

```
summary(mtcars) # Summary of data
```

```
##      mpg          cyl          disp          hp          drat          mi
## Min.   :10.40    Min.   :4.000    Min.   : 71.1    Min.   : 52.0    Min.   :2.760    Mi
## 1st Qu.:15.43    1st Qu.:4.000    1st Qu.:120.8    1st Qu.: 96.5    1st Qu.:3.080    1st
## Median :19.20    Median :6.000    Median :196.3    Median :123.0    Median :3.695    Med
## Mean   :20.09    Mean   :6.188    Mean   :230.7    Mean   :146.7    Mean   :3.597    Mea
## 3rd Qu.:22.80    3rd Qu.:8.000    3rd Qu.:326.0    3rd Qu.:180.0    3rd Qu.:3.920    3rd
## Max.   :33.90    Max.   :8.000    Max.   :472.0    Max.   :335.0    Max.   :4.930    Max

##      vs          am          gear          carb
## Min.   :0.0000    Min.   :0.0000    Min.   :3.000    Min.   :1.000
## 1st Qu.:0.0000    1st Qu.:0.0000    1st Qu.:3.000    1st Qu.:2.000
## Median :0.0000    Median :0.0000    Median :4.000    Median :2.000
## Mean   :0.4375    Mean   :0.4062    Mean   :3.688    Mean   :2.812
## 3rd Qu.:1.0000    3rd Qu.:1.0000    3rd Qu.:4.000    3rd Qu.:4.000
## Max.   :1.0000    Max.   :1.0000    Max.   :5.000    Max.   :8.000
```

You can also use the `View()` function to view the data in a spreadsheet like format.

```
View(mtcars)
```

Or you can use the `edit()` function to edit the data in a spreadsheet like format.

```
#edit(mtcars)
```

2.13.1 Names of the columns

```
names(mtcars)
```

```
## [1] "mpg" "cyl" "disp" "hp" "drat" "wt" "qsec" "vs" "am" "gear" "carb"
```

2.13.2 Number of rows and columns

```
dim(mtcars) # Number of rows and columns
```

```
## [1] 32 11
```

```
nrow(mtcars) # Number of rows
```

```
## [1] 32
```

```
ncol(mtcars) # Number of columns
```

```
## [1] 11
```

2.13.3 Accessing the data

```
mtcars[1, 1] # First row, first column
```

```
## [1] 21
```

```
mtcars[1, ] # First row, all columns
```

```
##           mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4  21   6  160 110  3.9 2.62 16.46  0  1    4    4
```

```
mtcars[, 1] # All rows, first column
```

```
## [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4 14.7 32.4
## [23] 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7 15.0 21.4
```

```
mtcars[1:5, ] # First five rows, all columns
```

```
##           mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160 110  3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21.0   6  160 110  3.90 2.875 17.02  0  1    4    4
## Datsun 710      22.8   4  108  93  3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive  21.4   6  258 110  3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175  3.15 3.440 17.02  0  0    3    2
```

```
mtcars[, 1:5] # All rows, first five columns
```

```
##           mpg cyl  disp  hp drat
## Mazda RX4      21.0   6 160.0 110 3.90
## Mazda RX4 Wag  21.0   6 160.0 110 3.90
## Datsun 710      22.8   4 108.0  93 3.85
## Hornet 4 Drive  21.4   6 258.0 110 3.08
## Hornet Sportabout 18.7   8 360.0 175 3.15
## Valiant         18.1   6 225.0 105 2.76
## Duster 360      14.3   8 360.0 245 3.21
## Merc 240D       24.4   4 146.7  62 3.69
## Merc 230        22.8   4 140.8  95 3.92
## Merc 280        19.2   6 167.6 123 3.92
## Merc 280C       17.8   6 167.6 123 3.92
## Merc 450SE      16.4   8 275.8 180 3.07
## Merc 450SL      17.3   8 275.8 180 3.07
## Merc 450SLC     15.2   8 275.8 180 3.07
## Cadillac Fleetwood 10.4   8 472.0 205 2.93
## Lincoln Continental 10.4   8 460.0 215 3.00
## Chrysler Imperial 14.7   8 440.0 230 3.23
## Fiat 128        32.4   4  78.7  66 4.08
```

```
## Honda Civic          30.4  4  75.7  52 4.93
## Toyota Corolla      33.9  4  71.1  65 4.22
## Toyota Corona       21.5  4 120.1  97 3.70
## Dodge Challenger    15.5  8 318.0 150 2.76
## AMC Javelin         15.2  8 304.0 150 3.15
## Camaro Z28          13.3  8 350.0 245 3.73
## Pontiac Firebird    19.2  8 400.0 175 3.08
## Fiat X1-9           27.3  4  79.0  66 4.08
## Porsche 914-2       26.0  4 120.3  91 4.43
## Lotus Europa        30.4  4  95.1 113 3.77
## Ford Pantera L      15.8  8 351.0 264 4.22
## Ferrari Dino        19.7  6 145.0 175 3.62
## Maserati Bora       15.0  8 301.0 335 3.54
## Volvo 142E         21.4  4 121.0 109 4.11
```

```
mtcars[1:5, 1:5] # First five rows, first five columns
```

```
##           mpg cyl disp  hp drat
## Mazda RX4      21.0   6  160 110 3.90
## Mazda RX4 Wag  21.0   6  160 110 3.90
## Datsun 710     22.8   4  108  93 3.85
## Hornet 4 Drive  21.4   6  258 110 3.08
## Hornet Sportabout 18.7   8  360 175 3.15
```

```
mtcars[1:5, c(1, 3, 5)] # First five rows, columns 1, 3 and 5
```

```
##           mpg disp drat
## Mazda RX4      21.0  160 3.90
## Mazda RX4 Wag  21.0  160 3.90
## Datsun 710     22.8  108 3.85
## Hornet 4 Drive  21.4  258 3.08
## Hornet Sportabout 18.7  360 3.15
```

```
mtcars[1:5, c("mpg", "disp", "hp")] # First five rows, columns mpg, disp and hp
```

```
##           mpg disp  hp
## Mazda RX4      21.0  160 110
## Mazda RX4 Wag  21.0  160 110
## Datsun 710     22.8  108  93
## Hornet 4 Drive  21.4  258 110
## Hornet Sportabout 18.7  360 175
```

2.13.4 Accessing the data using column names

Use the `$` operator to access the data using column names.

```
mtcars$mpg # Accessing the mpg column
```

```
## [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4
```



```
## [23] 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7 15.0 21.4
```

```
mtcars$mpg[1] # Accessing the first element of the mpg column
```

```
## [1] 21
```

```
mtcars$mpg[1:5] # Accessing the first five elements of the mpg column
```

```
## [1] 21.0 21.0 22.8 21.4 18.7
```

```
mtcars$mpg[mtcars$mpg > 20] # Accessing the elements of the mpg column that are greater than 20
```

```
## [1] 21.0 21.0 22.8 21.4 24.4 22.8 32.4 30.4 33.9 21.5 27.3 26.0 30.4 21.4
```


Chapter 3

Making your First Plot

Plots are commonly made using the `ggplot2` package. `ggplot` stands for grammar of graphics. It is a plotting system that is implemented in the `ggplot2` package. It is based on the book Grammar of Graphics by Leland Wilkinson. The plots are made up of different layers.

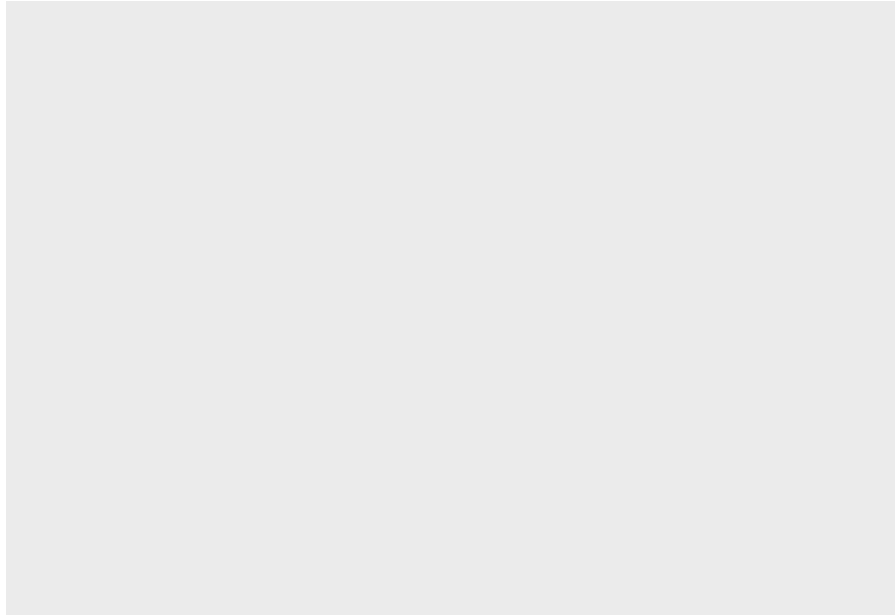
- The first layer is the data layer. It is specified using the `data` argument.
- The second layer is the aesthetic layer. It is specified using the `aes()` function.
- The third layer is the geometric layer. It is specified using the `geom_*`() functions.
- The fourth layer is the statistical layer. It is specified using the `stat_*`() functions.
- The fifth layer is the coordinate layer. It is specified using the `coord_*`() functions.
- The sixth layer is the facet layer. It is specified using the `facet_*`() functions.
- The seventh layer is the scale layer. It is specified using the `scale_*`() functions.
- The eighth layer is the label layer. It is specified using the `labs()` function.
- The ninth layer is the theme layer. It is specified using the `theme_*`() functions.

3.1 Loading the data

```
library(ggplot2)
data(mtcars)      # mt cars data set is included in the ggplot2 package
```

```
ggplot(data = mtcars)
```

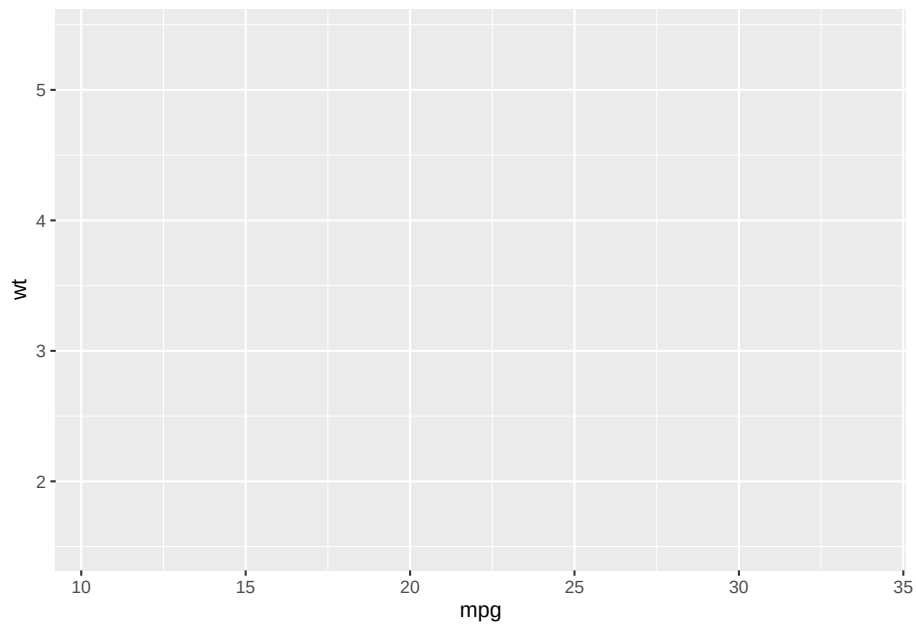
3.2 — **FOR THE DATA LAYER**



The plot is empty because we have not specified the aesthetic layer.

```
ggplot(data = mtcars, aes(x = mpg, y = wt))
```

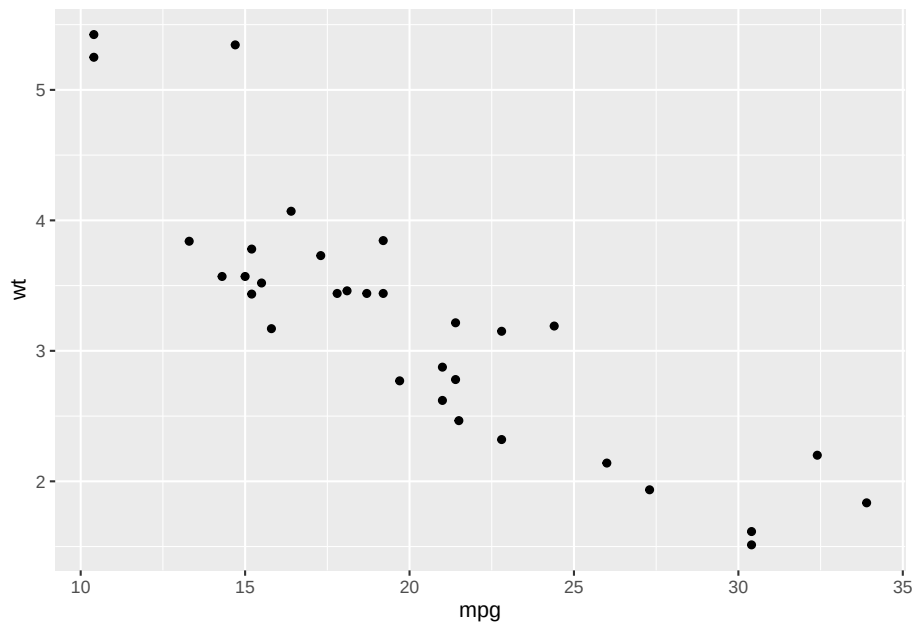
3.3 — **FOR THE AESTHETIC LAYER**



The plot is still empty but because we have specified the aesthetic layer, we can now there are x and y axes with labels `mpg` and `wt` available in the graph.

```
ggplot(data = mtcars, aes(x = mpg, y = wt)) +  
geom_point()
```

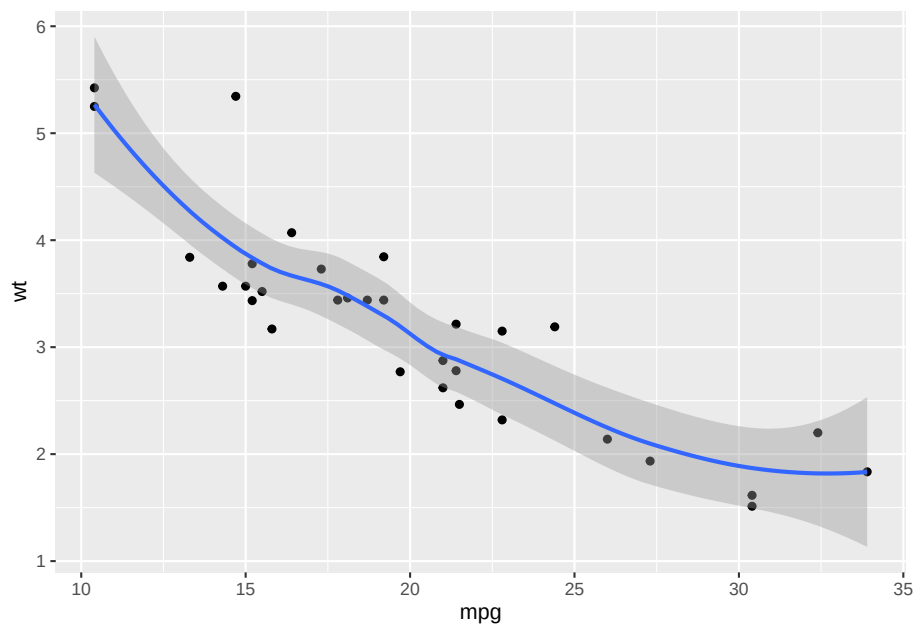
3.4. PLOT THE GEOMETRIC LAYER



The plot is now a scatter plot of mpg vs wt.

```
ggplot(data = mtcars, aes(x = mpg, y = wt)) +  
  geom_point() +  
  stat_smooth()
```

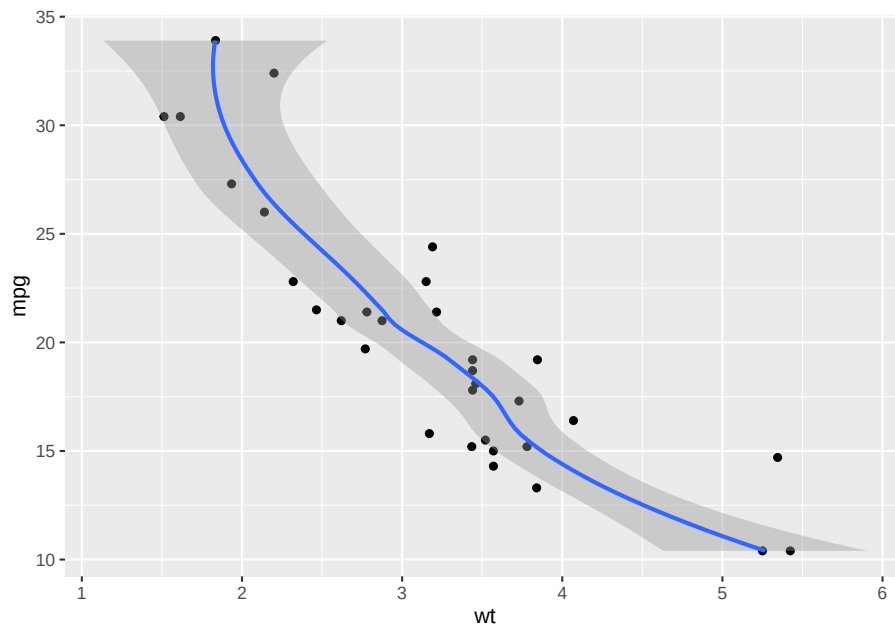
```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



The plot is now a scatter plot of `mpg` vs `wt` with a smooth line fit through the points.

```
ggplot(data = mtcars, aes(x = mpg, y = wt)) +  
  geom_point() +  
  stat_smooth() +  
  coord_flip()
```

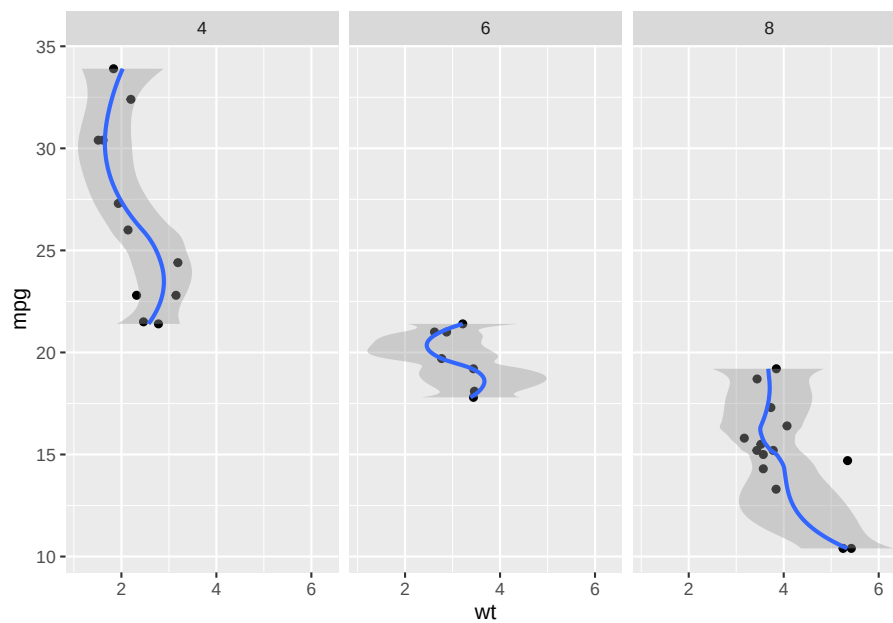
```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



The plot is now a scatter plot of `mpg` vs `wt` with a smooth line fit through the points and the axes have been flipped.

```
ggplot(data = mtcars, aes(x = mpg, y = wt)) +  
  geom_point() +  
  stat_smooth() +  
  coord_flip() +  
  facet_wrap(~ cyl)
```

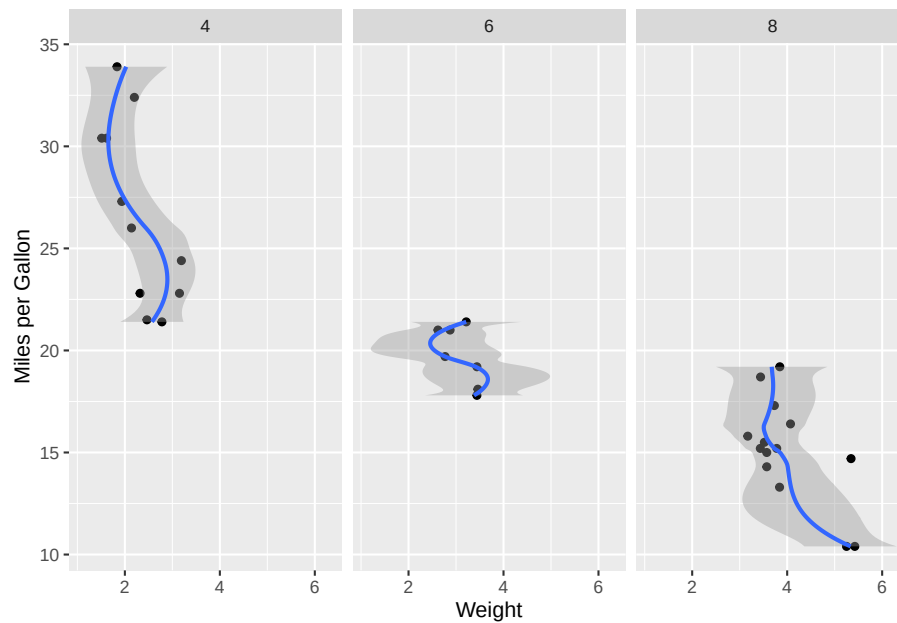
```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

The plot is now a scatter plot of `mpg` vs `wt` with a smooth line fit through the points and the axes have been flipped. The data has been faceted by the number of cylinders.

```
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  stat_smooth() +
  coord_flip() +
  facet_wrap(~ cyl) +
  scale_x_continuous(name = "Miles per Gallon") +
  scale_y_continuous(name = "Weight")
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



The plot is now a scatter plot of `mpg` vs `wt` with a smooth line fit through the points and the axes have been flipped. The data has been faceted by the number of cylinders. The axes have been labeled.

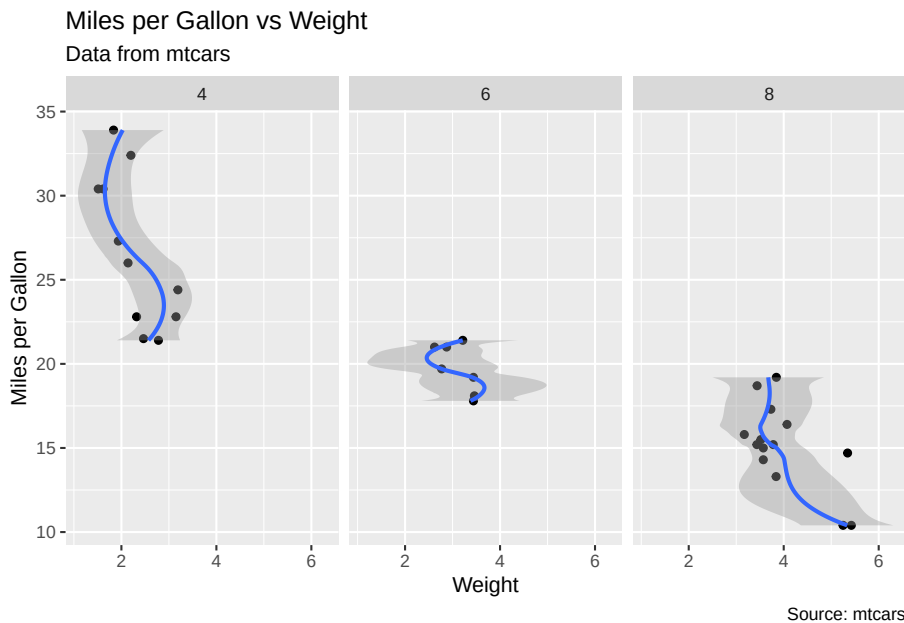
```

3.3 Plotting with ggplot2

ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  stat_smooth() +
  coord_flip() +
  facet_wrap(~ cyl) +
  scale_x_continuous(name = "Miles per Gallon") +
  scale_y_continuous(name = "Weight") +
  labs(title = "Miles per Gallon vs Weight", subtitle = "Data from mtcars", caption = "Source: mtcars")

## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'

```



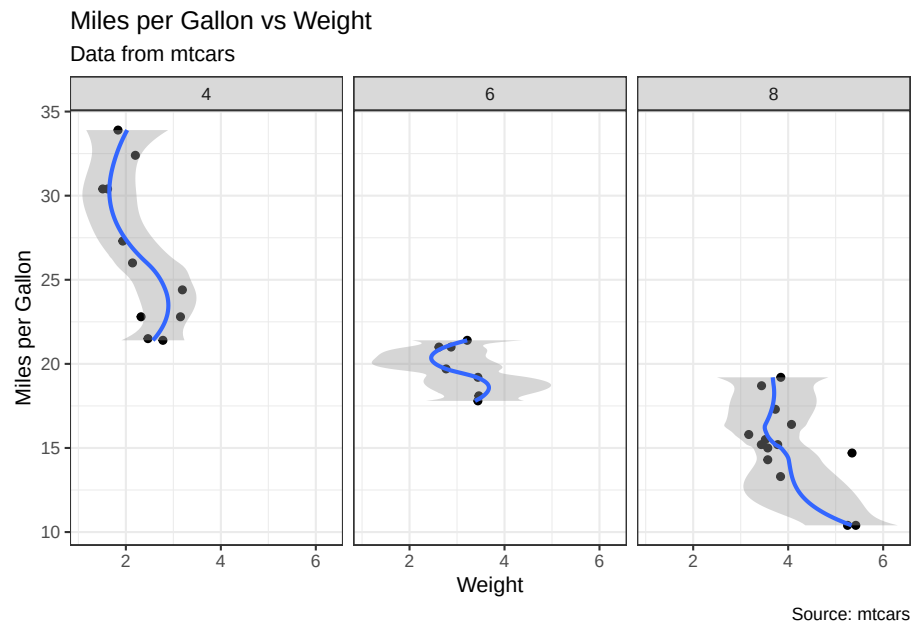
The plot is now a scatter plot of `mpg` vs `wt` with a smooth line fit through the points and the axes have been flipped. The data has been faceted by the number of cylinders. The axes have been labeled. The title, subtitle and caption have been added.

3.10 Plot the Theme Layer

There are several themes available in the `ggplot2` package. The default theme is `theme_grey()`. The other themes are `theme_bw()`, `theme_classic()`, `theme_dark()`, `theme_light()`, `theme_linedraw()`, `theme_minimal()`, `theme_void()`, `theme_test()`. You can also create your own theme using the `theme()` function. I recommend testing out the different themes to see which one you like best.

```
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  stat_smooth() +
  coord_flip() +
  facet_wrap(~ cyl) +
  scale_x_continuous(name = "Miles per Gallon") +
  scale_y_continuous(name = "Weight") +
  labs(title = "Miles per Gallon vs Weight", subtitle = "Data from mtcars", caption = "Source: mtcars") +
  theme_bw()
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



The plot is now a scatter plot of `mpg` vs `wt` with a smooth line fit through the points and the axes have been flipped. The data has been faceted by the number of cylinders. The axes have been labeled. The title, subtitle and caption have been added. The theme has been changed to black and white.

Chapter 4

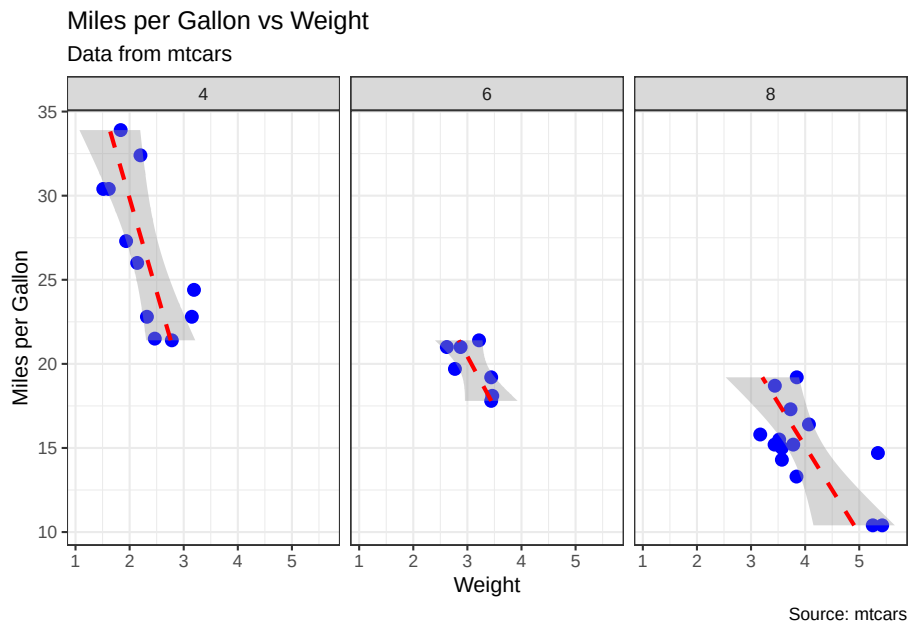
Additional Customizations

4.1 Customizing Aesthetics

You can further customize your plots by modifying various elements such as colors, point shapes, line types, and more. Here are a few examples:

```
ggplot(data = mtcars, aes(x = mpg, y = wt)) +  
  geom_point(color = "blue", shape = 16, size = 3) +  
  stat_smooth(method = "lm", color = "red", linetype = "dashed") +  
  coord_flip() +  
  facet_wrap(~ cyl) +  
  scale_x_continuous(name = "Miles per Gallon") +  
  scale_y_continuous(name = "Weight") +  
  labs(title = "Miles per Gallon vs Weight", subtitle = "Data from mtcars", caption = "Source: mt  
  theme_bw()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



The plot is now a scatter plot of `mpg` vs `wt` with blue points and a red dashed line fit through the points. The axes have been flipped. The data has been faceted by the number of cylinders. The axes have been labeled. The title, subtitle and caption have been added. The theme has been changed to black and white.

4.2 Combining Multiple Plots

You can combine multiple plots into a single figure using the `patchwork` package. Here's an example:

```
library(patchwork)
```

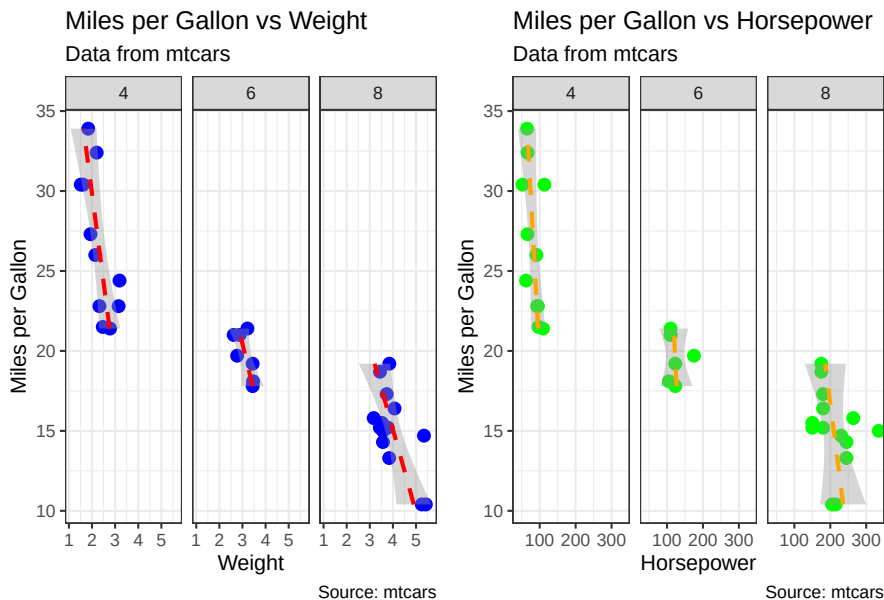
```
p1 <- ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point(color = "blue", shape = 16, size = 3) +
  stat_smooth(method = "lm", color = "red", linetype = "dashed") +
  coord_flip() +
  facet_wrap(~ cyl) +
  scale_x_continuous(name = "Miles per Gallon") +
  scale_y_continuous(name = "Weight") +
  labs(title = "Miles per Gallon vs Weight", subtitle = "Data from mtcars", caption = "Source: mtcars") +
  theme_bw()

p2 <- ggplot(data = mtcars, aes(x = mpg, y = hp)) +
  geom_point(color = "green", shape = 16, size = 3) +
  stat_smooth(method = "lm", color = "orange", linetype = "dashed") +
```

```
coord_flip() +
facet_wrap(~ cyl) +
scale_x_continuous(name = "Miles per Gallon") +
scale_y_continuous(name = "Horsepower") +
labs(title = "Miles per Gallon vs Horsepower", subtitle = "Data from mtcars", caption = "Source: mtcars") +
theme_bw()

p1 + p2
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```



The above code creates two separate plots, `p1` and `p2`, and combines them side by side using the `+` operator from the `patchwork` package.

4.3 Interactive Plots

There are many options to create interactive plots in R. Popular packages include `plotly`, `ggiraph`, and `shiny`. Here's a simple example using the `plotly` package to convert a `ggplot` into an interactive plot:

```
library(plotly)

p <- ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
```

```

stat_smooth() +
coord_flip() +
facet_wrap(~ cyl) +
scale_x_continuous(name = "Miles per Gallon") +
scale_y_continuous(name = "Weight") +
labs(title = "Miles per Gallon vs Weight", subtitle = "Data from mtcars", caption = " ")
theme_bw()

ggplotly(p)

```

`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'

The above code creates an interactive version of the ggplot using the `ggplotly()` function from the `plotly` package. You can hover over points to see their values and zoom in/out on the plot.

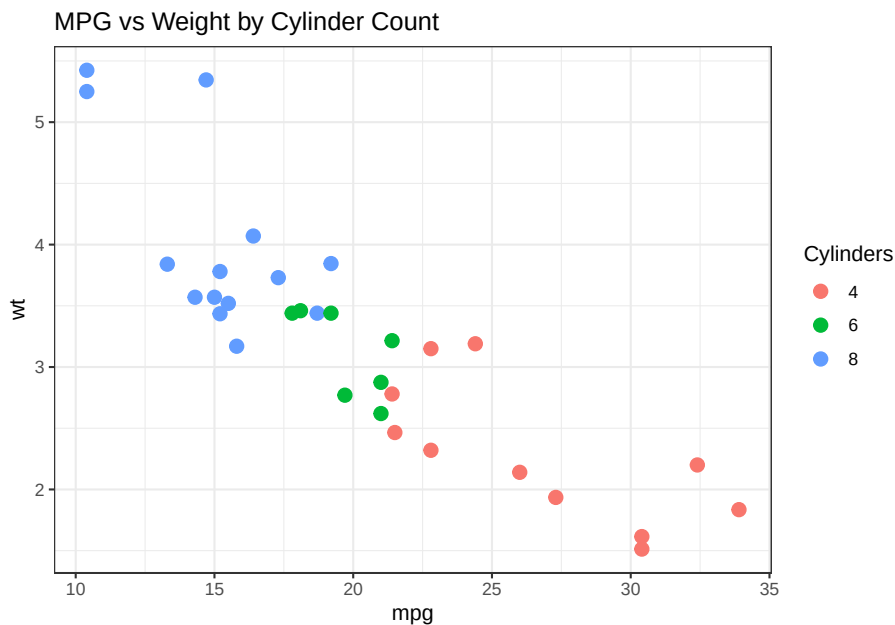
4.4 Color and Fill Mappings

You can map colors to variables in your data to visualize additional dimensions:

```

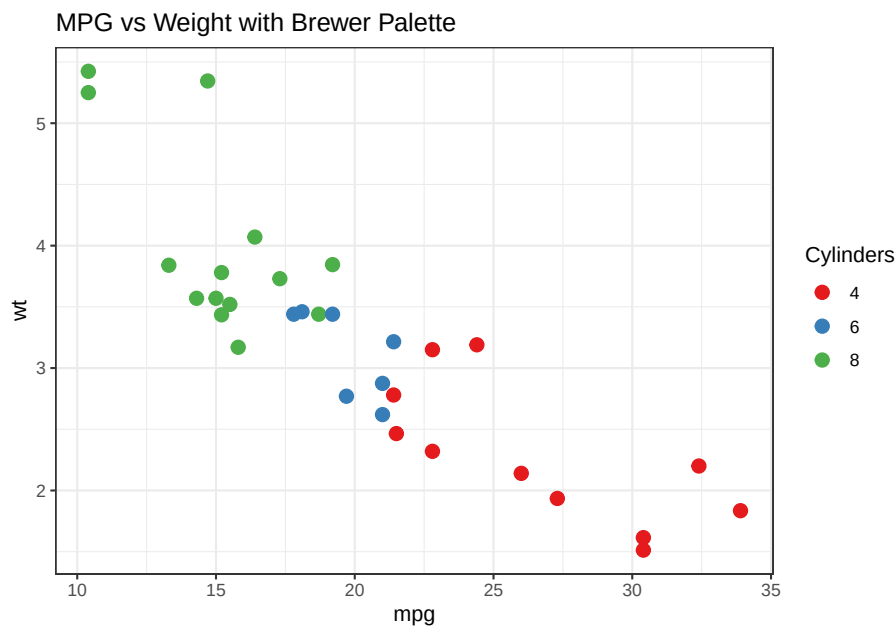
# Mapping color to a variable
ggplot(data = mtcars, aes(x = mpg, y = wt, color = factor(cyl))) +
  geom_point(size = 3) +
  labs(title = "MPG vs Weight by Cylinder Count", color = "Cylinders") +
  theme_bw()

```

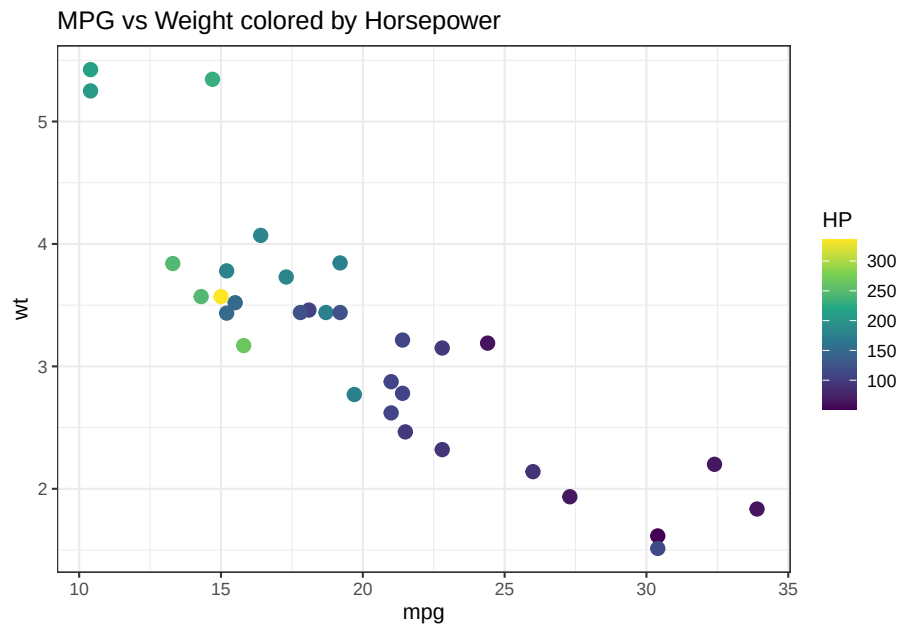



```
# Using different color palettes
library(RColorBrewer)

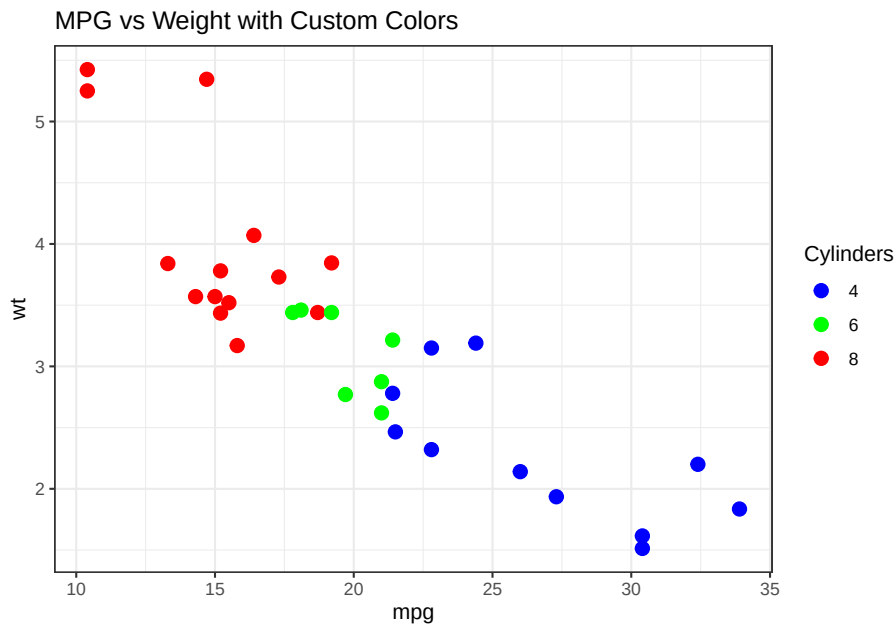
ggplot(data = mtcars, aes(x = mpg, y = wt, color = factor(cyl))) +
  geom_point(size = 3) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "MPG vs Weight with Brewer Palette", color = "Cylinders") +
  theme_bw()
```



```
# Using viridis color palette (colorblind-friendly)
ggplot(data = mtcars, aes(x = mpg, y = wt, color = hp)) +
  geom_point(size = 3) +
  scale_color_viridis_c() +
  labs(title = "MPG vs Weight colored by Horsepower", color = "HP") +
  theme_bw()
```



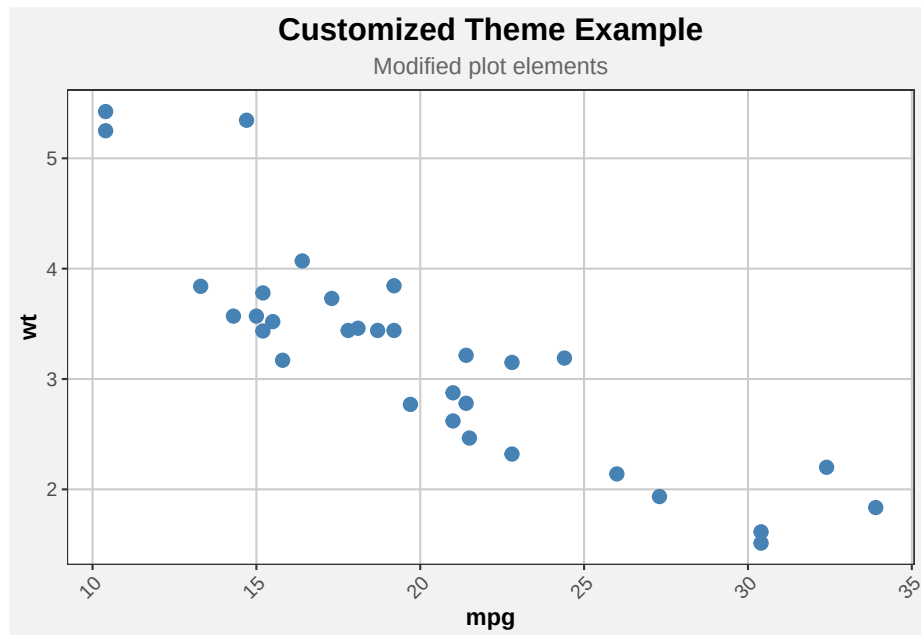
```
# Manual color assignment
ggplot(data = mtcars, aes(x = mpg, y = wt, color = factor(cyl))) +
  geom_point(size = 3) +
  scale_color_manual(values = c("4" = "blue", "6" = "green", "8" = "red")) +
  labs(title = "MPG vs Weight with Custom Colors", color = "Cylinders") +
  theme_bw()
```



4.5 Advanced Theme Customization

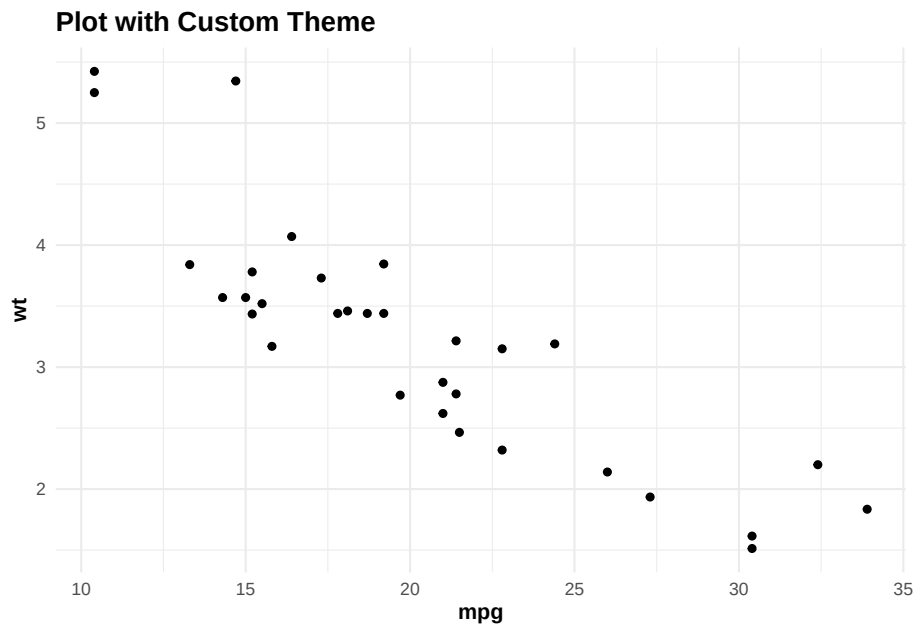
You can customize virtually every aspect of your plot using the `theme()` function:

```
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point(color = "steelblue", size = 3) +
  labs(title = "Customized Theme Example",
        subtitle = "Modified plot elements") +
  theme_bw() +
  theme(
    plot.title = element_text(size = 16, face = "bold", hjust = 0.5),
    plot.subtitle = element_text(size = 12, hjust = 0.5, color = "gray40"),
    axis.title = element_text(size = 12, face = "bold"),
    axis.text = element_text(size = 10),
    axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid.major = element_line(color = "gray80"),
    panel.grid.minor = element_blank(),
    panel.background = element_rect(fill = "white"),
    plot.background = element_rect(fill = "gray95")
  )
```



```
# Creating and saving a custom theme
my_custom_theme <- theme_minimal() +
  theme(
    plot.title = element_text(size = 14, face = "bold"),
    axis.title = element_text(size = 11, face = "bold"),
    legend.position = "bottom"
  )

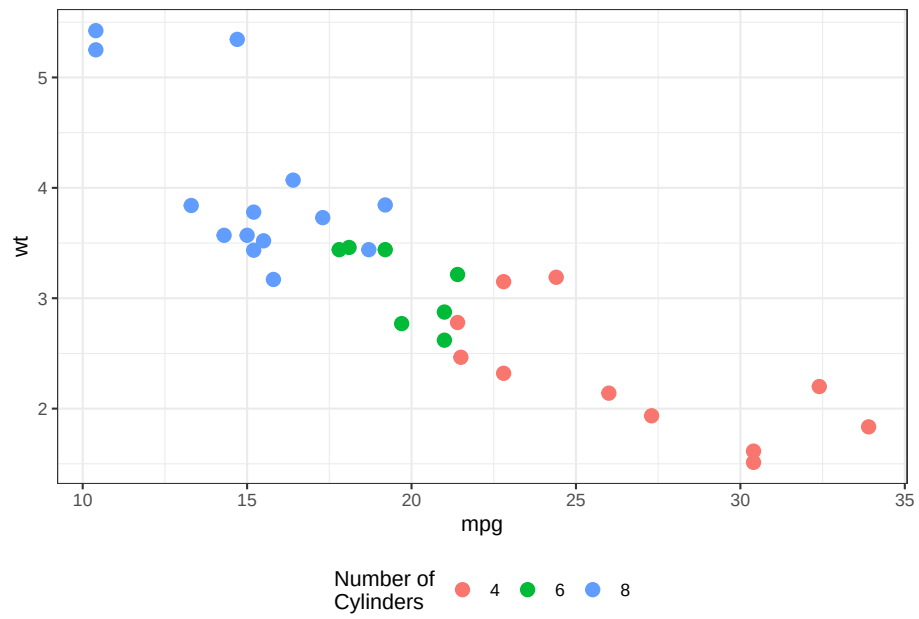
# Apply custom theme to any plot
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  labs(title = "Plot with Custom Theme") +
  my_custom_theme
```



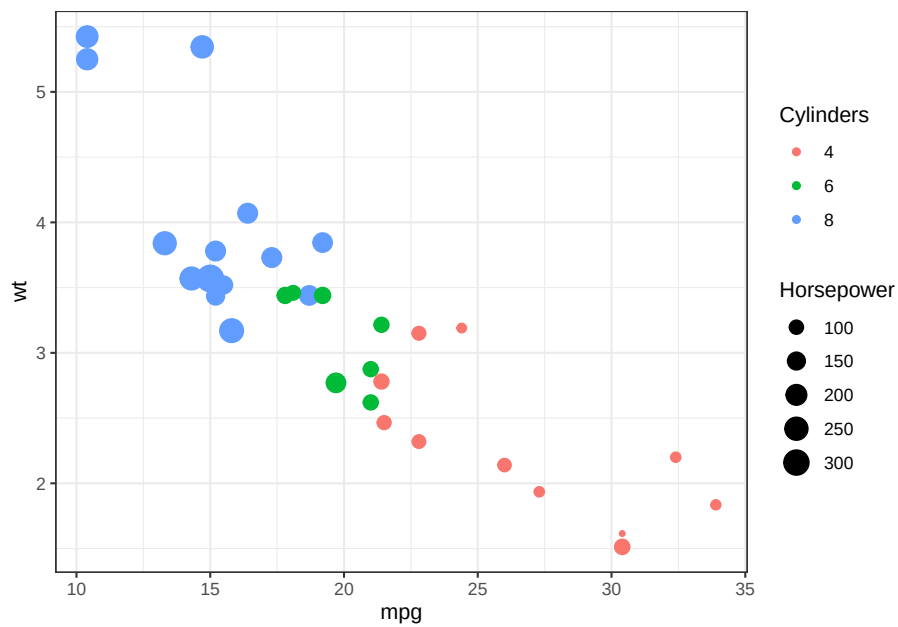
4.6 Legend Customization

Control the appearance and position of legends in your plots:

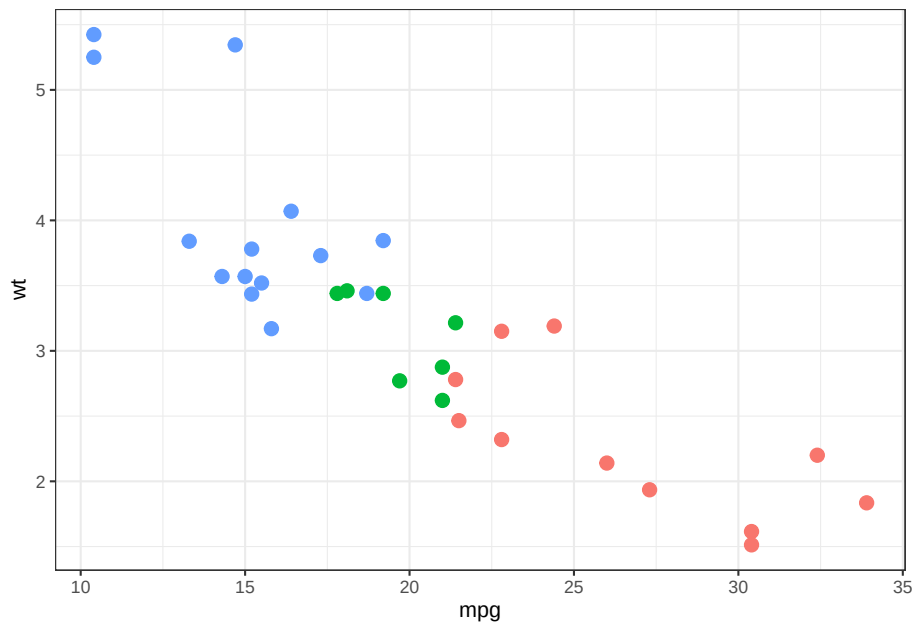
```
# Repositioning legend
ggplot(data = mtcars, aes(x = mpg, y = wt, color = factor(cyl))) +
  geom_point(size = 3) +
  labs(color = "Number of\nCylinders") +
  theme_bw() +
  theme(legend.position = "bottom") # Options: "top", "left", "right", "none"
```



```
# Fine-tuning legend with guides()
ggplot(data = mtcars, aes(x = mpg, y = wt, color = factor(cyl), size = hp)) +
  geom_point() +
  guides(
    color = guide_legend(title = "Cylinders", order = 1),
    size = guide_legend(title = "Horsepower", order = 2)
  ) +
  theme_bw()
```



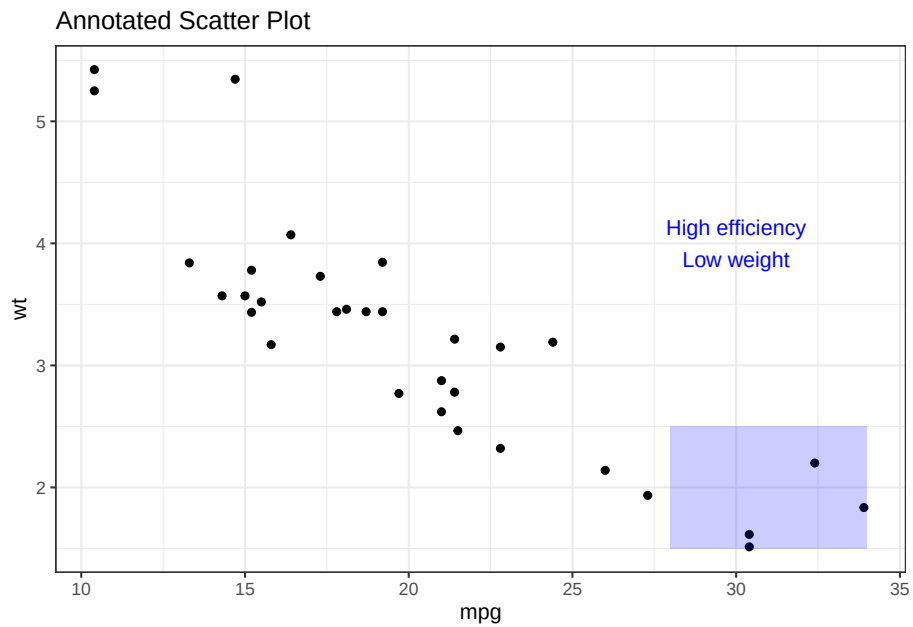
```
# Removing legend
ggplot(data = mtcars, aes(x = mpg, y = wt, color = factor(cyl))) +
  geom_point(size = 3) +
  theme_bw() +
  theme(legend.position = "none")
```



4.7 Annotations

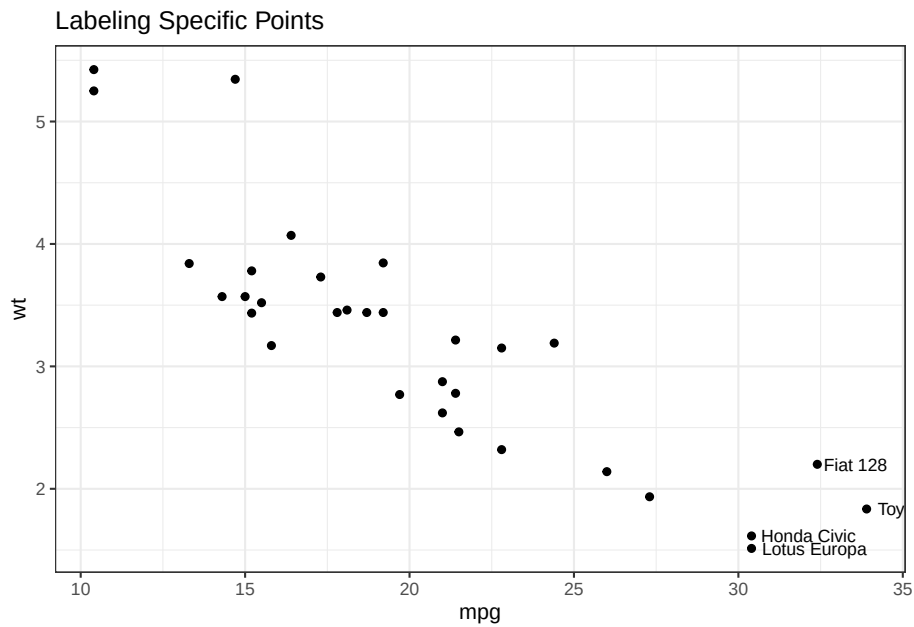
Add text, shapes, and reference lines to highlight important features:

```
# Adding text annotations
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  annotate("text", x = 30, y = 4, label = "High efficiency\nLow weight",
    color = "blue", size = 4) +
  annotate("rect", xmin = 28, xmax = 34, ymin = 1.5, ymax = 2.5,
    alpha = 0.2, fill = "blue") +
  labs(title = "Annotated Scatter Plot") +
  theme_bw()
```

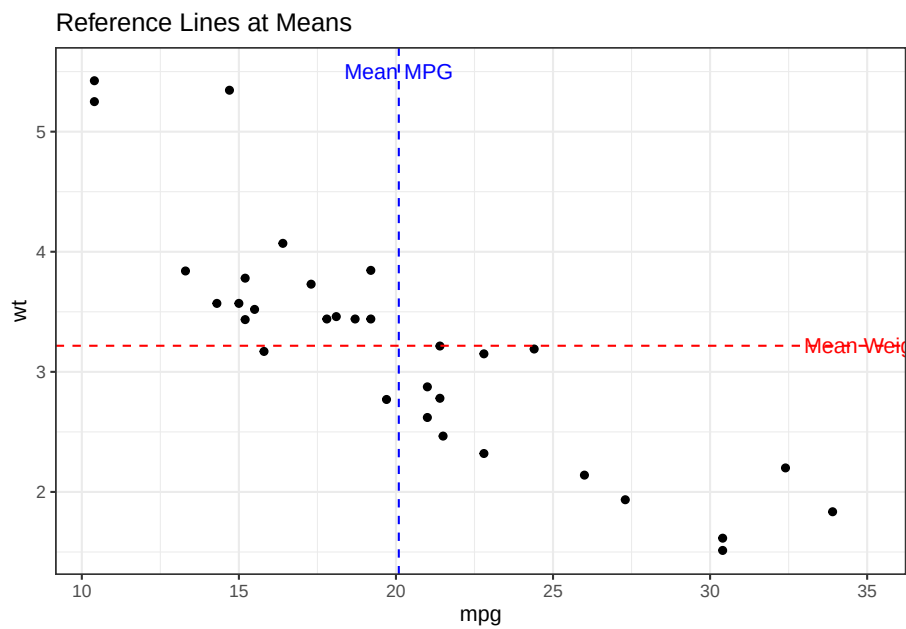



```
# Labeling specific points
library(dplyr)
mtcars_labeled <- mtcars %>%
  mutate(car_name = rownames(mtcars))

ggplot(data = mtcars_labeled, aes(x = mpg, y = wt)) +
  geom_point() +
  geom_text(data = filter(mtcars_labeled, mpg > 30),
            aes(label = car_name, hjust = -0.1, size = 3)) +
  labs(title = "Labeling Specific Points") +
  theme_bw()
```



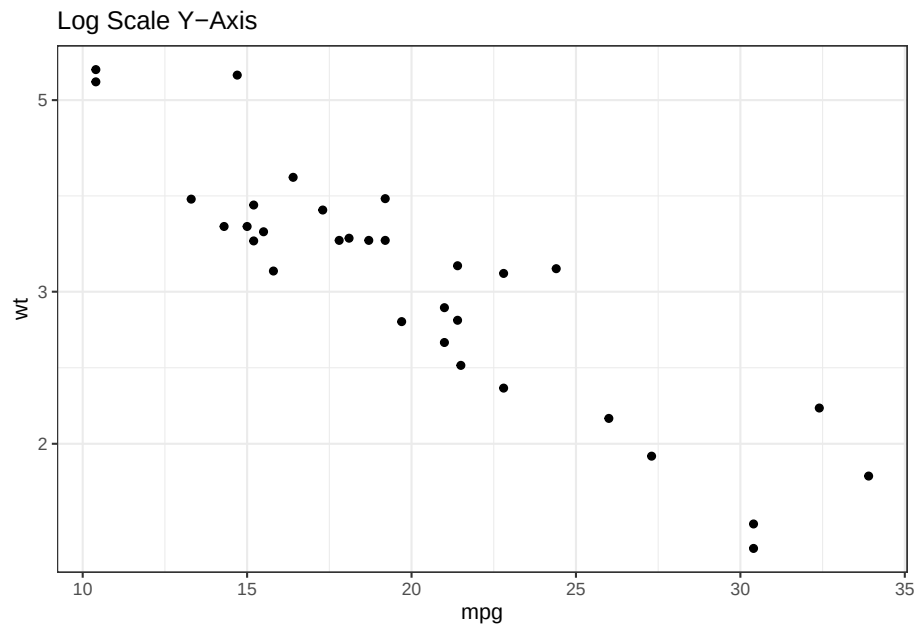
```
# Adding reference lines
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  geom_hline(yintercept = mean(mtcars$wt), linetype = "dashed", color = "red") +
  geom_vline(xintercept = mean(mtcars$mpg), linetype = "dashed", color = "blue") +
  annotate("text", x = mean(mtcars$mpg), y = 5.5, label = "Mean MPG", color = "blue") +
  annotate("text", x = 35, y = mean(mtcars$wt), label = "Mean Weight", color = "red") +
  labs(title = "Reference Lines at Means") +
  theme_bw()
```



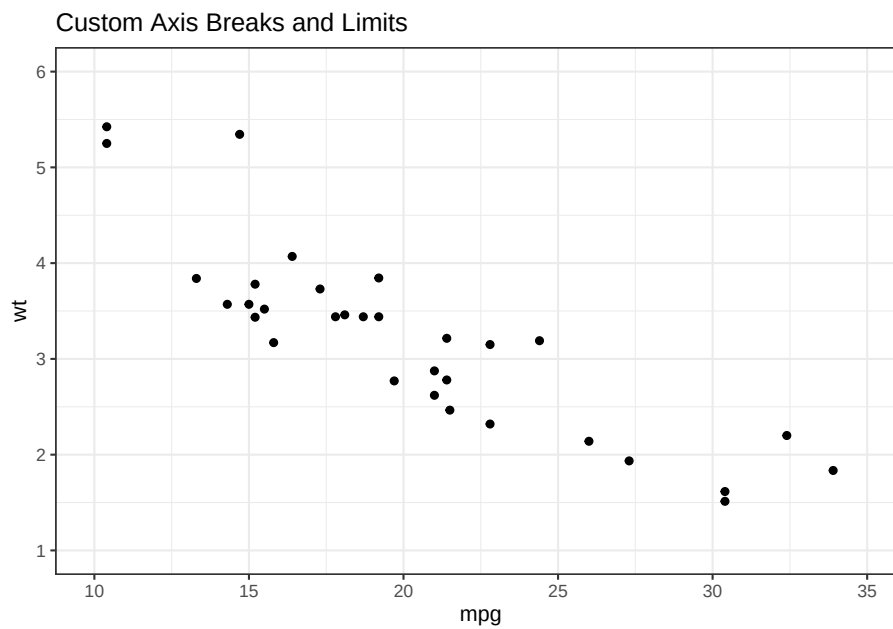
4.8 Axis Customization

Customize axes for better data representation:

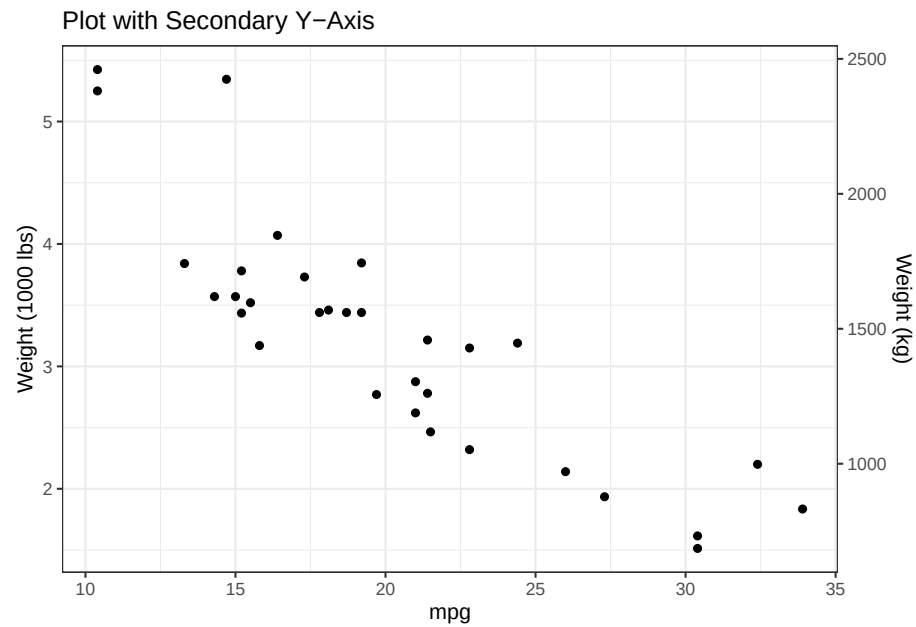
```
# Log scale transformation
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  scale_y_log10() +
  labs(title = "Log Scale Y-Axis") +
  theme_bw()
```



```
# Custom breaks and limits
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  scale_x_continuous(breaks = seq(10, 35, by = 5), limits = c(10, 35)) +
  scale_y_continuous(breaks = seq(1, 6, by = 1), limits = c(1, 6)) +
  labs(title = "Custom Axis Breaks and Limits") +
  theme_bw()
```

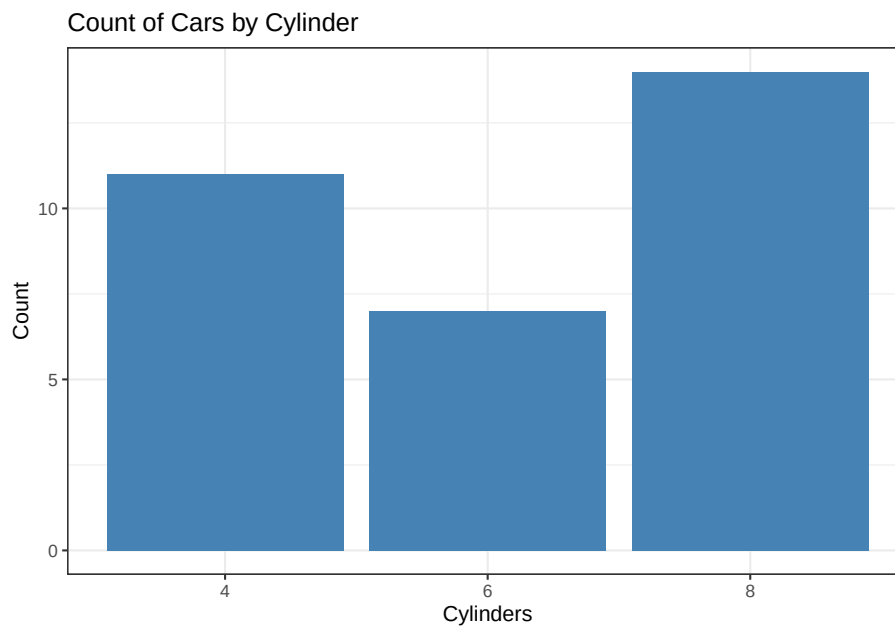


```
# Secondary axis
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  scale_y_continuous(
    name = "Weight (1000 lbs)",
    sec.axis = sec_axis(~ . * 453.592, name = "Weight (kg)")
  ) +
  labs(title = "Plot with Secondary Y-Axis") +
  theme_bw()
```



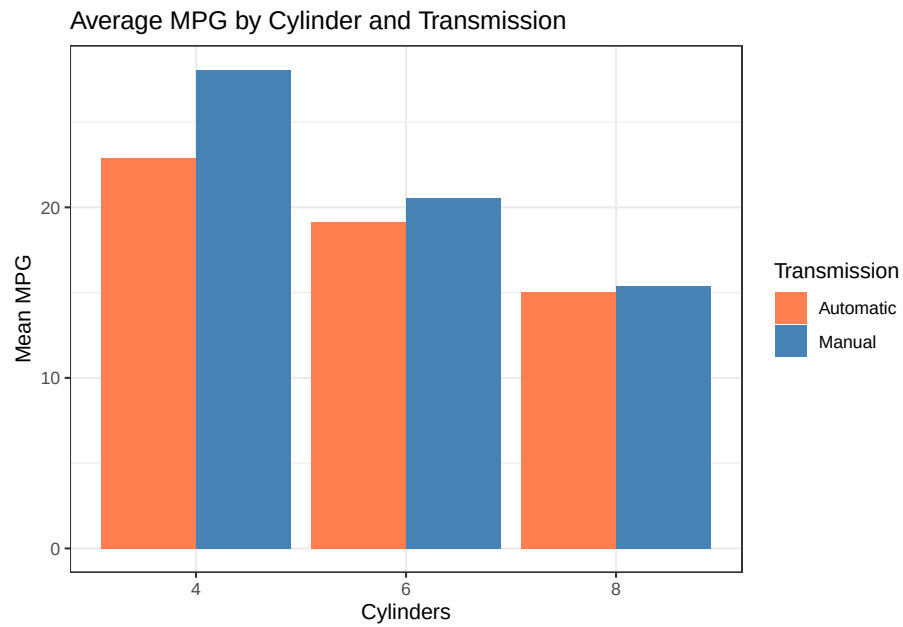
4.9 Different Plot Types

```
# Simple bar plot
ggplot(data = mtcars, aes(x = factor(cyl))) +
  geom_bar(fill = "steelblue") +
  labs(title = "Count of Cars by Cylinder", x = "Cylinders", y = "Count") +
  theme_bw()
```

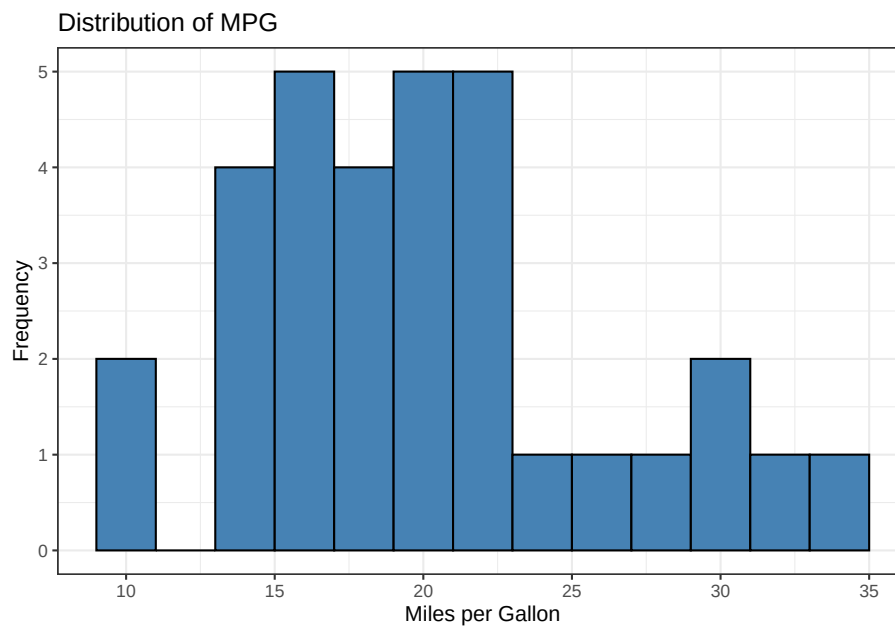


```
# Grouped bar plot
mtcars_summary <- mtcars %>%
  group_by(cyl, am) %>%
  summarize(mean_mpg = mean(mpg), .groups = "drop")

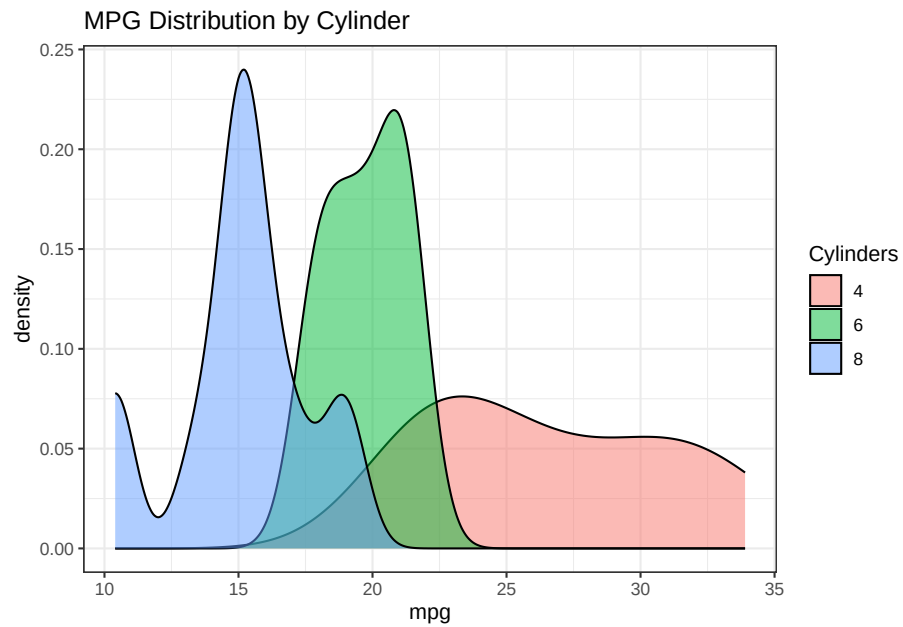
ggplot(data = mtcars_summary, aes(x = factor(cyl), y = mean_mpg, fill = factor(am))) +
  geom_col(position = "dodge") +
  labs(title = "Average MPG by Cylinder and Transmission",
       x = "Cylinders", y = "Mean MPG", fill = "Transmission") +
  scale_fill_manual(values = c("0" = "coral", "1" = "steelblue"),
                    labels = c("0" = "Automatic", "1" = "Manual")) +
  theme_bw()
```



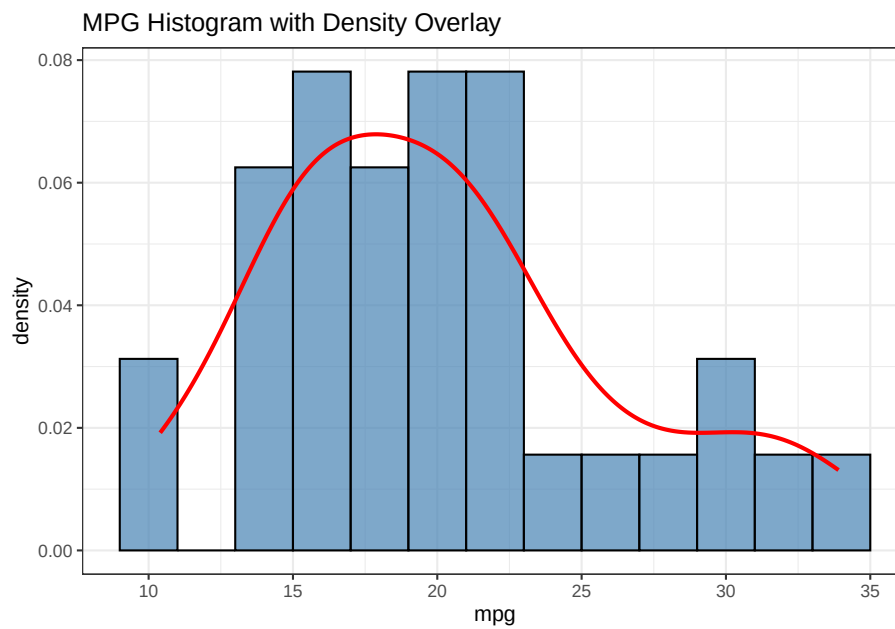
```
# Histogram
ggplot(data = mtcars, aes(x = mpg)) +
  geom_histogram(binwidth = 2, fill = "steelblue", color = "black") +
  labs(title = "Distribution of MPG", x = "Miles per Gallon", y = "Frequency") +
  theme_bw()
```

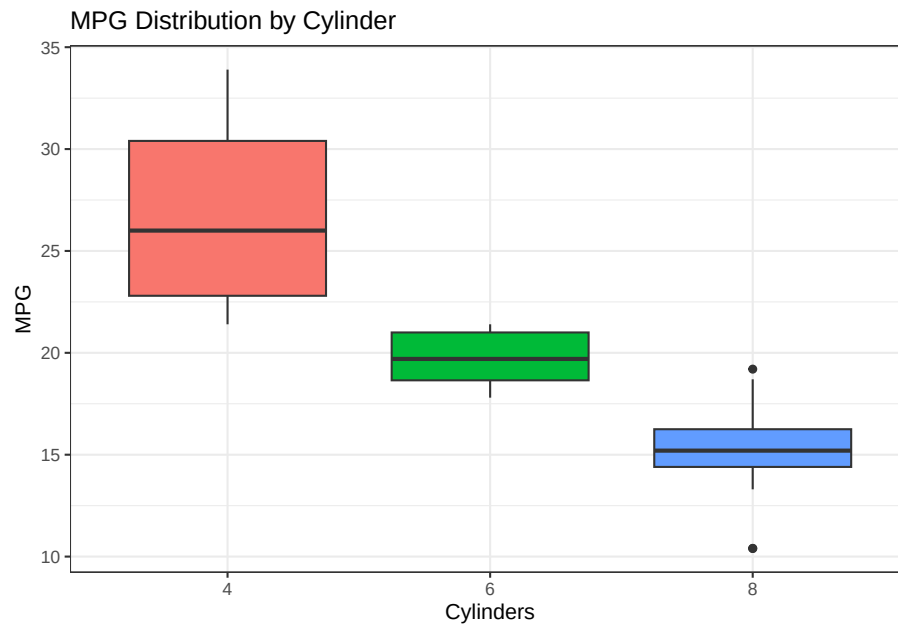
```
# Density plot
ggplot(data = mtcars, aes(x = mpg, fill = factor(cyl))) +
  geom_density(alpha = 0.5) +
  labs(title = "MPG Distribution by Cylinder", fill = "Cylinders") +
  theme_bw()
```



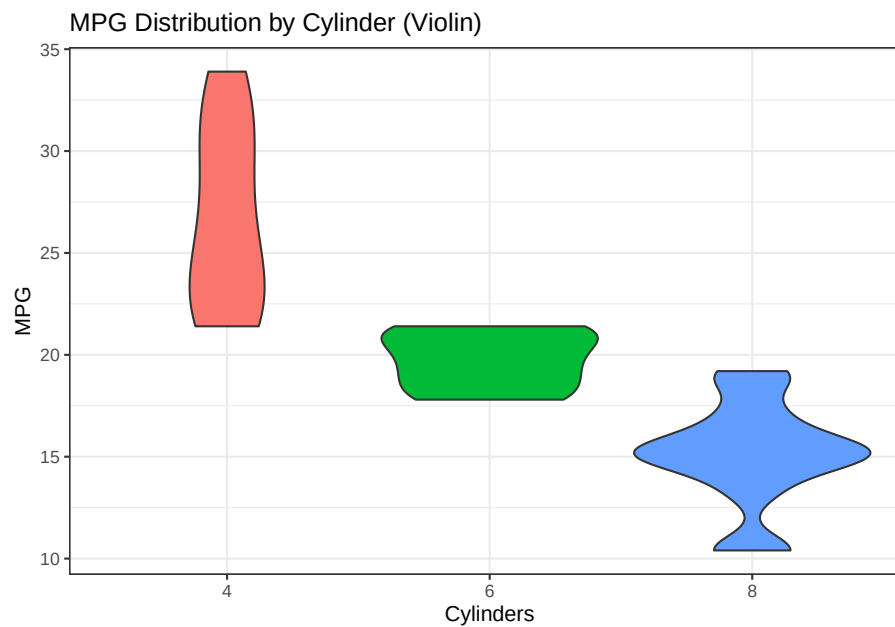
```
# Histogram with density overlay
ggplot(data = mtcars, aes(x = mpg)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 2,
    fill = "steelblue", color = "black", alpha = 0.7) +
  geom_density(color = "red", linewidth = 1) +
  labs(title = "MPG Histogram with Density Overlay") +
  theme_bw()
```



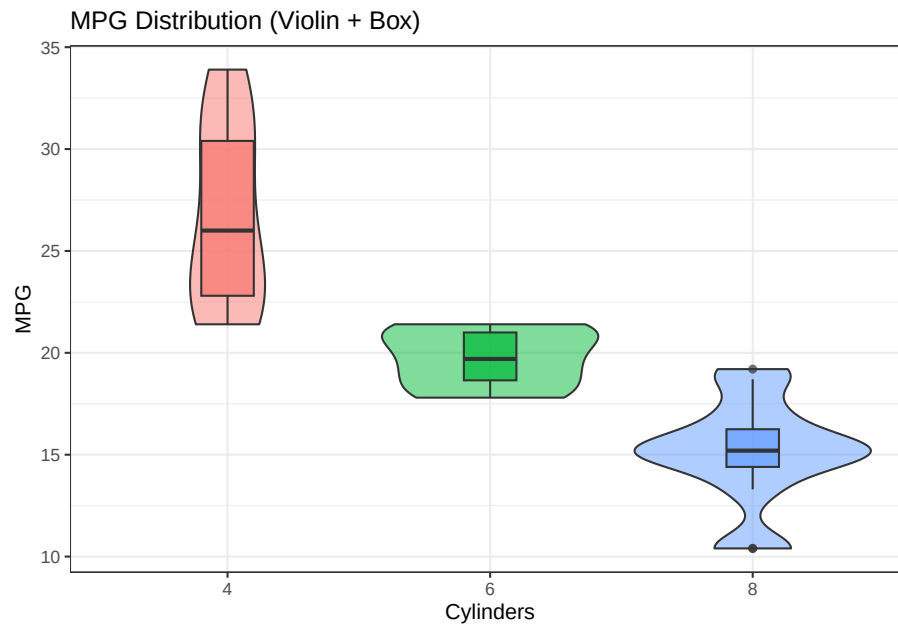
```
# Box plot
ggplot(data = mtcars, aes(x = factor(cyl), y = mpg, fill = factor(cyl))) +
  geom_boxplot() +
  labs(title = "MPG Distribution by Cylinder", x = "Cylinders", y = "MPG") +
  theme_bw() +
  theme(legend.position = "none")
```



```
# Violin plot
ggplot(data = mtcars, aes(x = factor(cyl), y = mpg, fill = factor(cyl))) +
  geom_violin() +
  labs(title = "MPG Distribution by Cylinder (Violin)", x = "Cylinders", y = "MPG") +
  theme_bw() +
  theme(legend.position = "none")
```

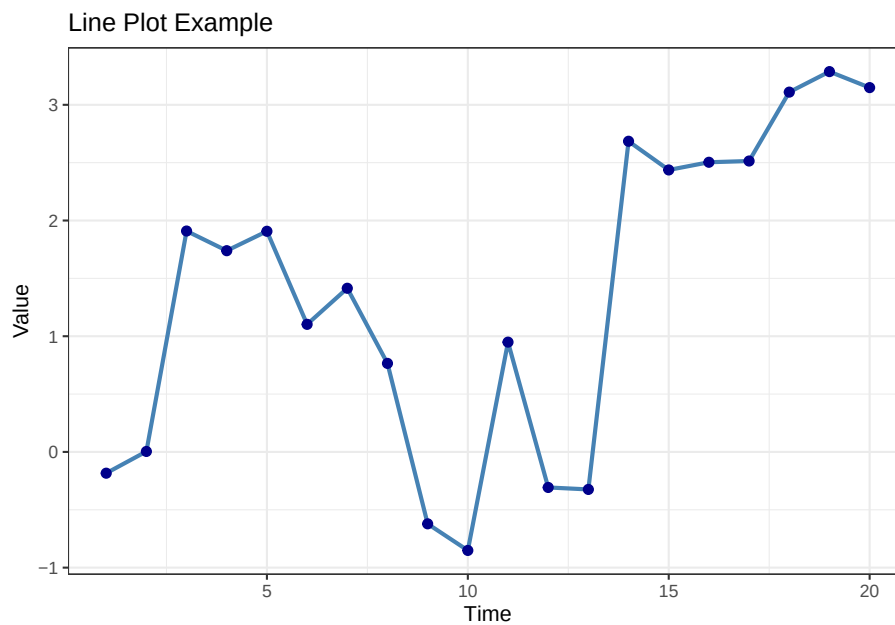


```
# Combined box and violin plot
ggplot(data = mtcars, aes(x = factor(cyl), y = mpg, fill = factor(cyl))) +
  geom_violin(alpha = 0.5) +
  geom_boxplot(width = 0.2, alpha = 0.7) +
  labs(title = "MPG Distribution (Violin + Box)", x = "Cylinders", y = "MPG") +
  theme_bw() +
  theme(legend.position = "none")
```



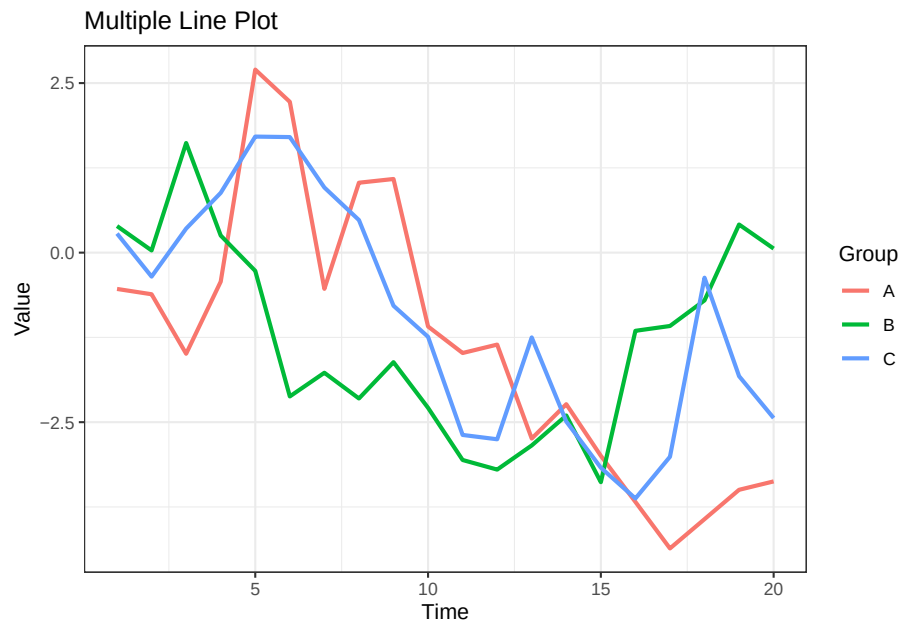
```
# Line plot (using different dataset for time series)
df_time <- data.frame(
  time = 1:20,
  value = cumsum(rnorm(20))
)

ggplot(data = df_time, aes(x = time, y = value)) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_point(color = "darkblue", size = 2) +
  labs(title = "Line Plot Example", x = "Time", y = "Value") +
  theme_bw()
```



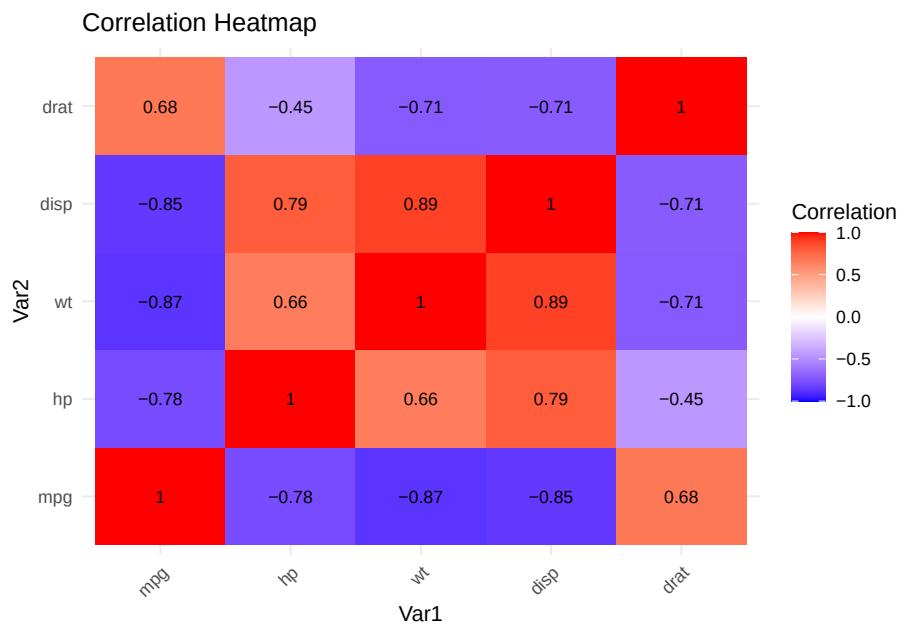
```
# Multiple lines
df_multi <- data.frame(
  time = rep(1:20, 3),
  value = c(cumsum(rnorm(20)), cumsum(rnorm(20)), cumsum(rnorm(20))),
  group = rep(c("A", "B", "C"), each = 20)
)

ggplot(data = df_multi, aes(x = time, y = value, color = group)) +
  geom_line(linewidth = 1) +
  labs(title = "Multiple Line Plot", x = "Time", y = "Value", color = "Group") +
  theme_bw()
```



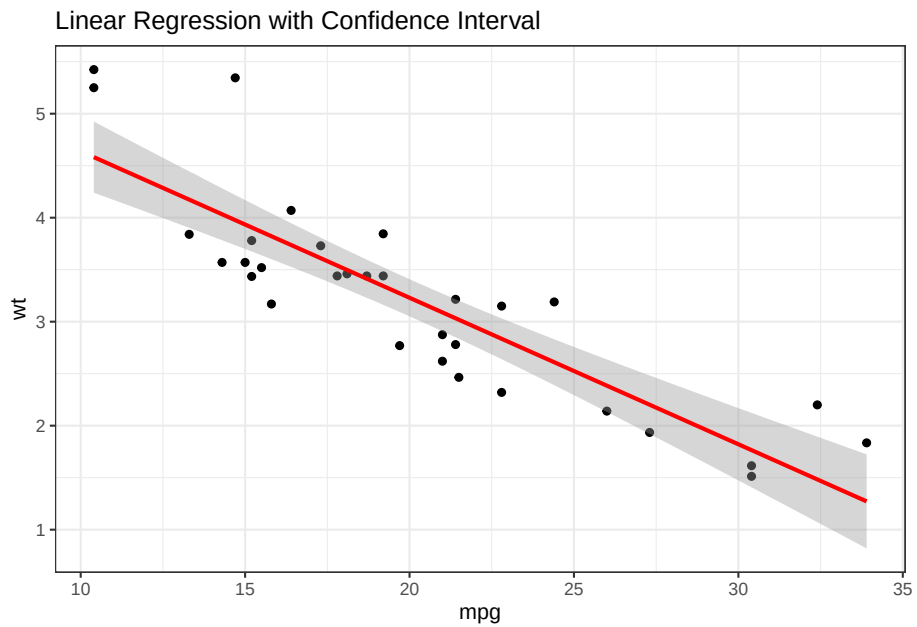
```
# Create correlation matrix
cor_matrix <- cor(mtcars[, c("mpg", "hp", "wt", "disp", "drat")])
cor_df <- as.data.frame(as.table(cor_matrix))
names(cor_df) <- c("Var1", "Var2", "Correlation")

ggplot(data = cor_df, aes(x = Var1, y = Var2, fill = Correlation)) +
  geom_tile() +
  scale_fill_gradient2(low = "blue", mid = "white", high = "red",
    midpoint = 0, limits = c(-1, 1)) +
  geom_text(aes(label = round(Correlation, 2)), size = 3) +
  labs(title = "Correlation Heatmap") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

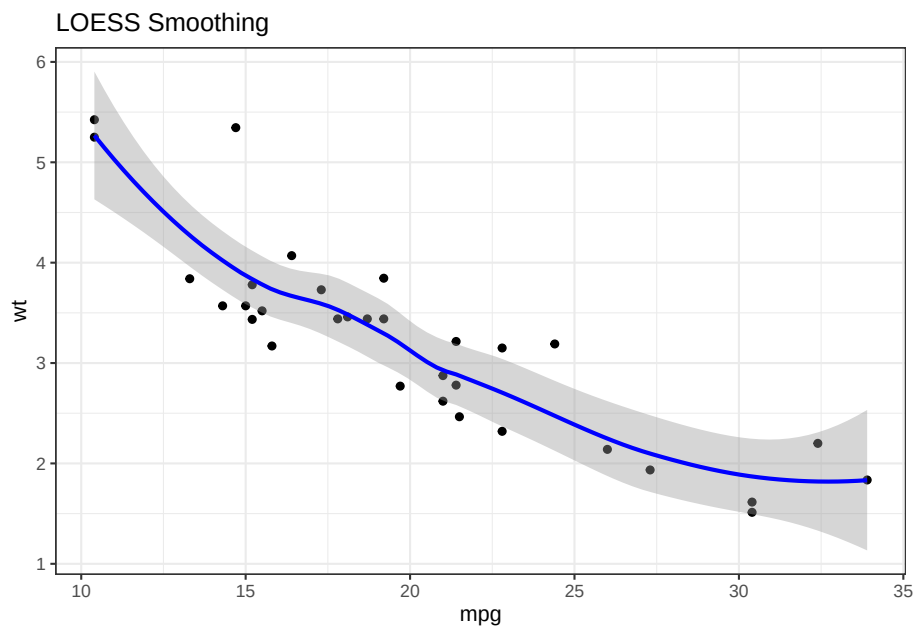
```
# Different smoothing methods
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  stat_smooth(method = "lm", color = "red", se = TRUE) + # Linear model
  labs(title = "Linear Regression with Confidence Interval") +
  theme_bw()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

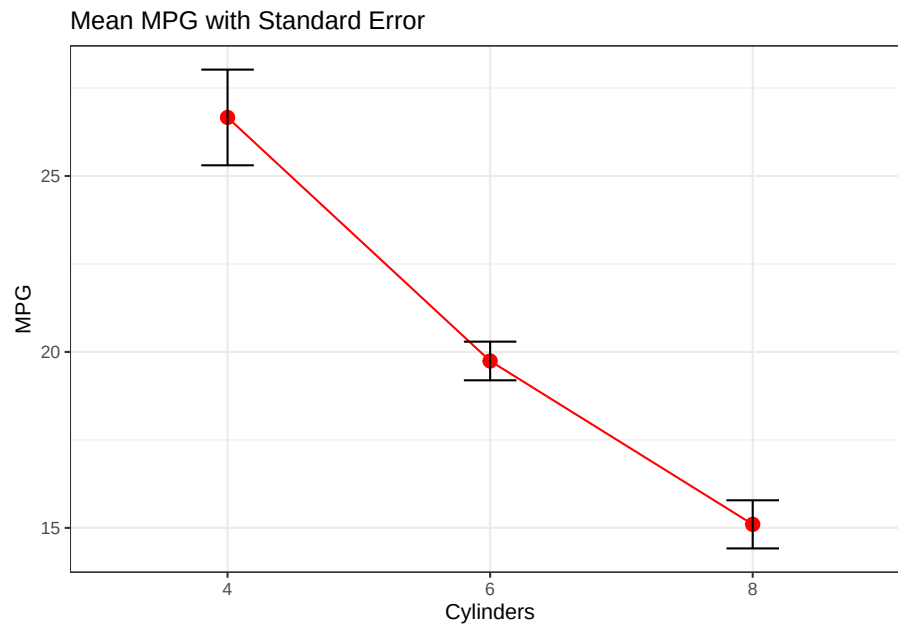


```
ggplot(data = mtcars, aes(x = mpg, y = wt)) +  
  geom_point() +  
  stat_smooth(method = "loess", color = "blue", se = TRUE) + # LOESS smoothing  
  labs(title = "LOESS Smoothing") +  
  theme_bw()
```

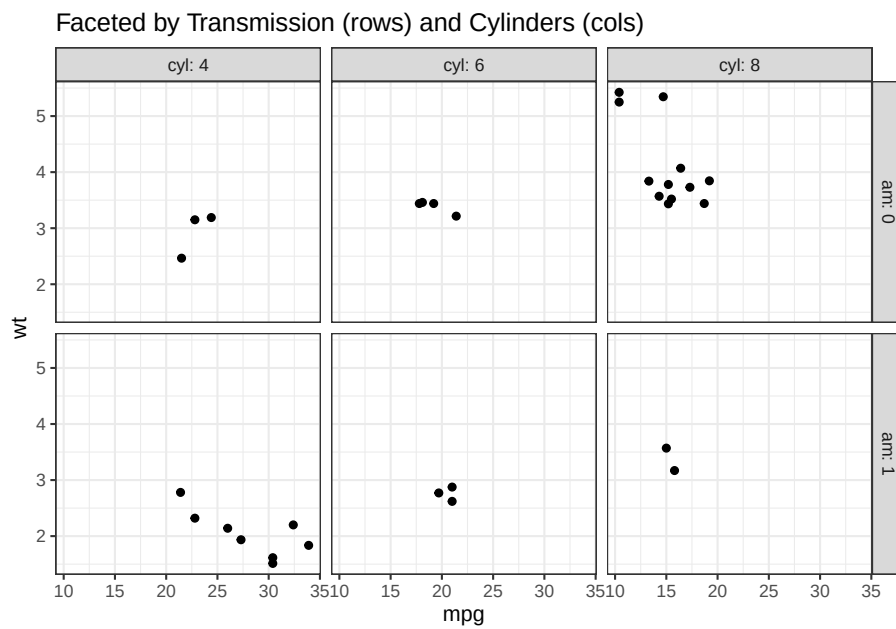
```
## `geom_smooth()` using formula = 'y ~ x'
```



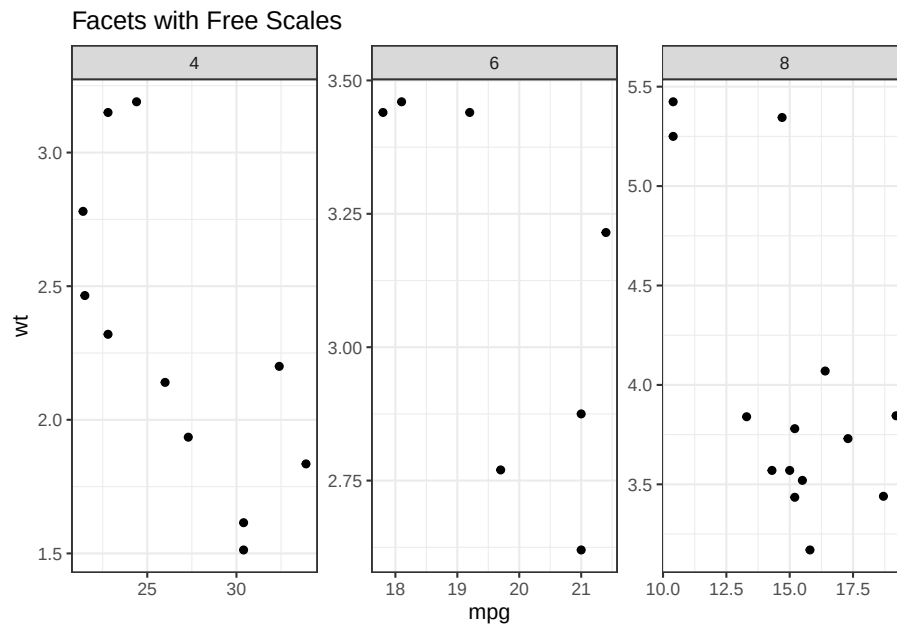
```
# Custom statistical summary
ggplot(data = mtcars, aes(x = factor(cyl), y = mpg)) +
  stat_summary(fun = mean, geom = "point", size = 3, color = "red") +
  stat_summary(fun = mean, geom = "line", aes(group = 1), color = "red") +
  stat_summary(fun.data = mean_se, geom = "errorbar", width = 0.2) +
  labs(title = "Mean MPG with Standard Error", x = "Cylinders", y = "MPG") +
  theme_bw()
```



```
# facet_grid for two variables
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  facet_grid(am ~ cyl, labeller = label_both) +
  labs(title = "Faceted by Transmission (rows) and Cylinders (cols)") +
  theme_bw()
```

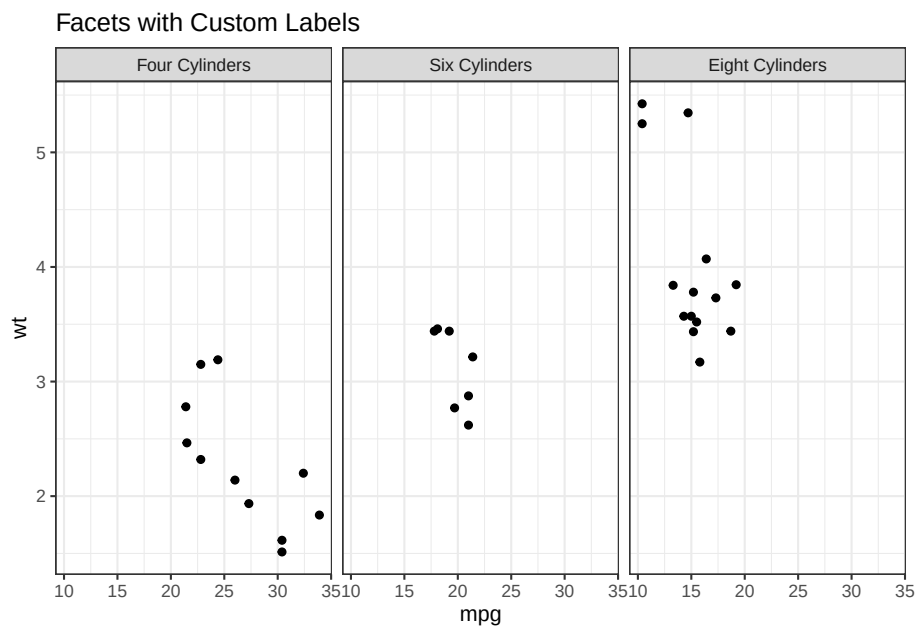


```
# Free scales in facets
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  facet_wrap(~ cyl, scales = "free") + # "free_x", "free_y", or "free"
  labs(title = "Facets with Free Scales") +
  theme_bw()
```

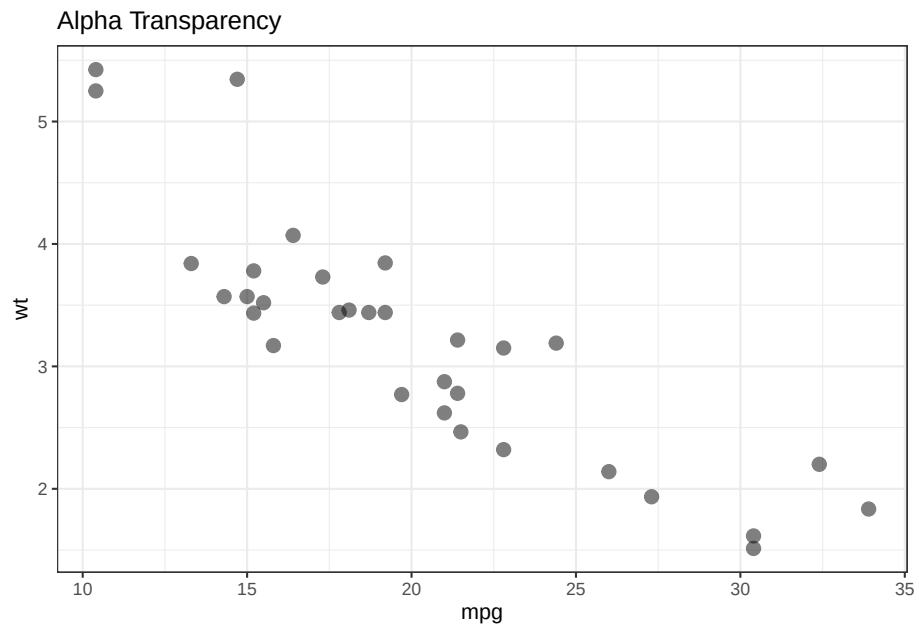


```
# Custom facet labels
cyl_labels <- c("4" = "Four Cylinders", "6" = "Six Cylinders", "8" = "Eight Cylinders")

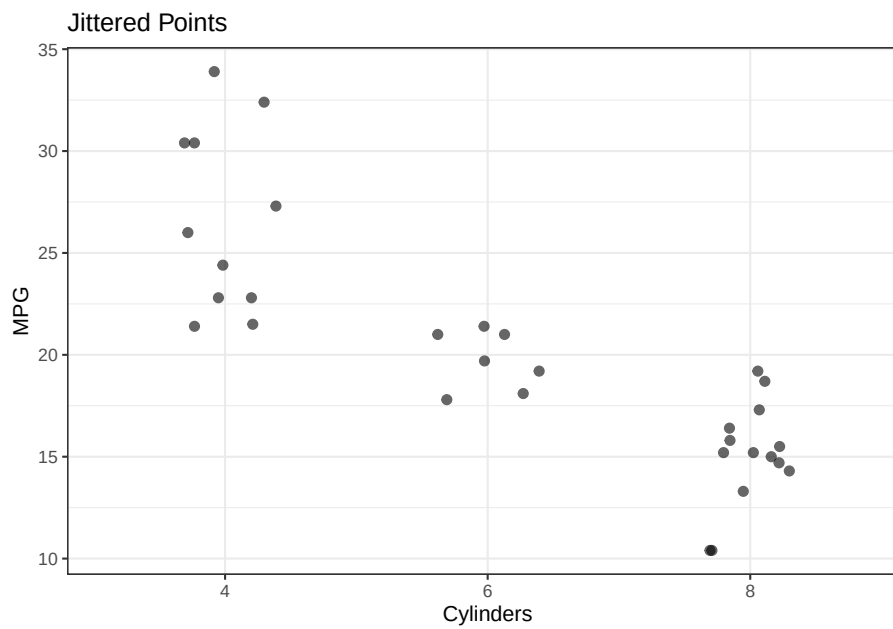
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point() +
  facet_wrap(~ cyl, labeller = labeller(cyl = cyl_labels)) +
  labs(title = "Facets with Custom Labels") +
  theme_bw()
```



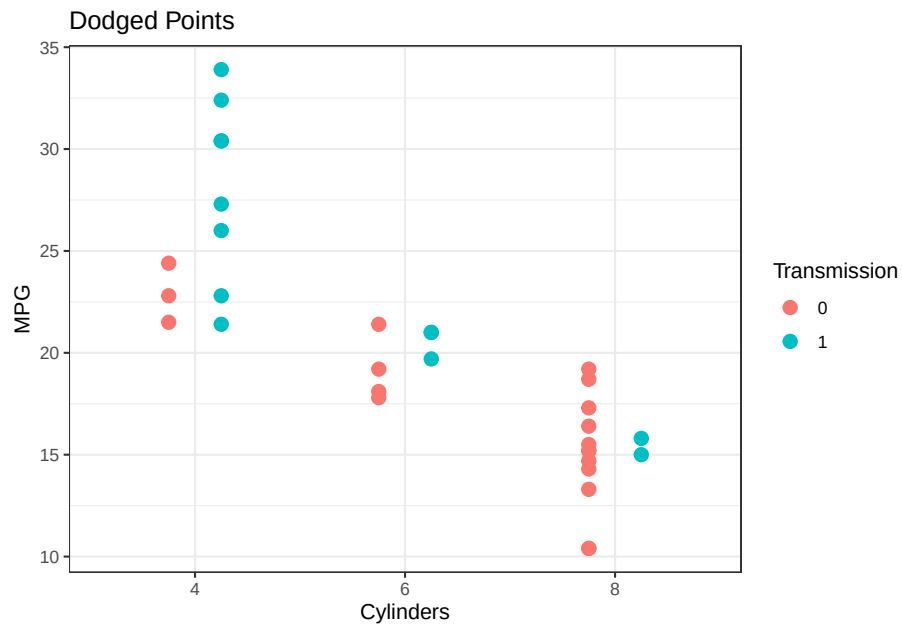
```
# Using alpha transparency
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point(alpha = 0.5, size = 3) +
  labs(title = "Alpha Transparency") +
  theme_bw()
```



```
# Jittering points
ggplot(data = mtcars, aes(x = factor(cyl), y = mpg)) +
  geom_jitter(width = 0.2, height = 0, alpha = 0.6, size = 2) +
  labs(title = "Jittered Points", x = "Cylinders", y = "MPG") +
  theme_bw()
```

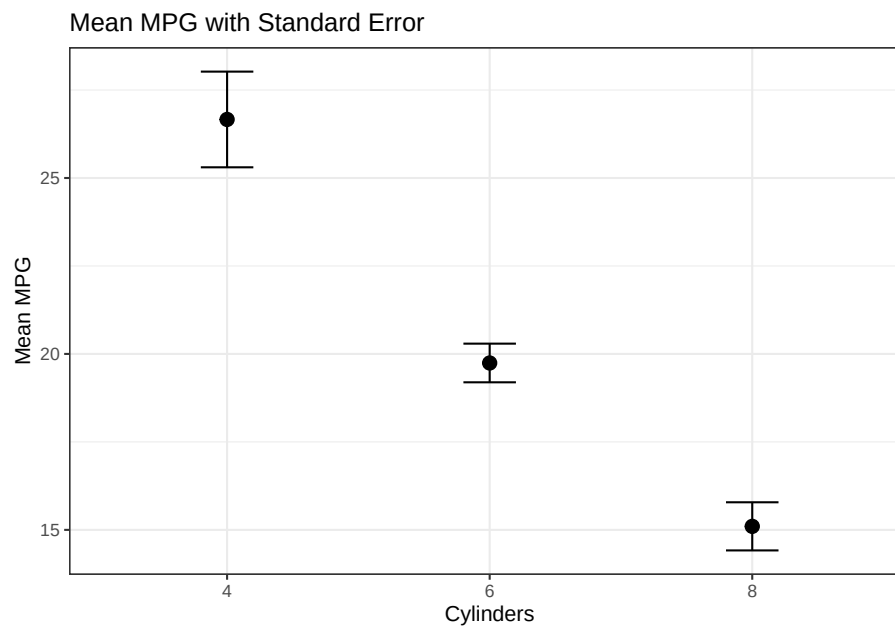



```
# Position dodge for grouped data
ggplot(data = mtcars, aes(x = factor(cyl), y = mpg, color = factor(am))) +
  geom_point(position = position_dodge(width = 0.5), size = 3) +
  labs(title = "Dodged Points", x = "Cylinders", y = "MPG", color = "Transmission") +
  theme_bw()
```



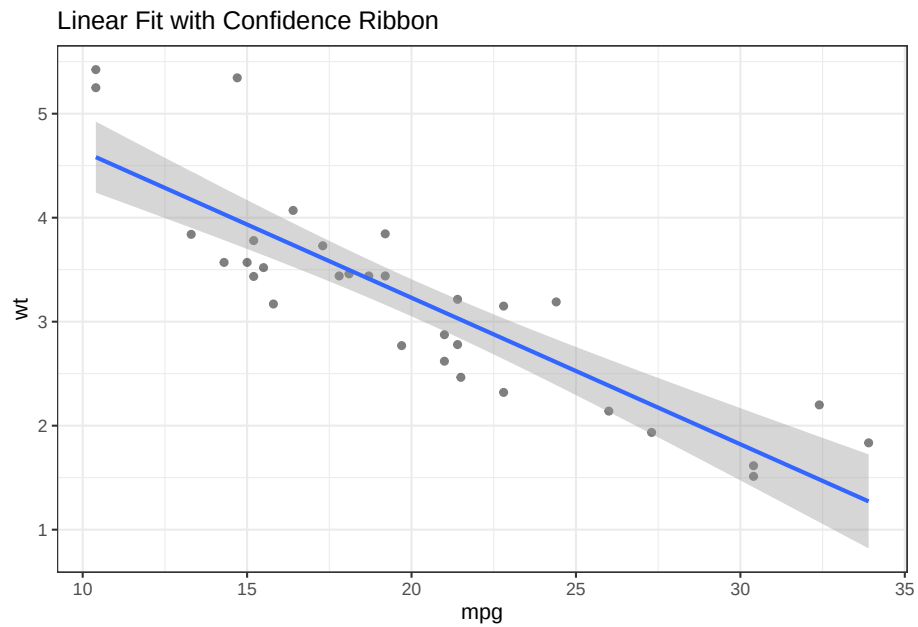
```
# Calculate summary statistics
mtcars_summary <- mtcars %>%
  group_by(cyl) %>%
  summarize(
    mean_mpg = mean(mpg),
    se_mpg = sd(mpg) / sqrt(n()),
    .groups = "drop"
  )

# Error bars
ggplot(data = mtcars_summary, aes(x = factor(cyl), y = mean_mpg)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_mpg - se_mpg, ymax = mean_mpg + se_mpg),
    width = 0.2) +
  labs(title = "Mean MPG with Standard Error",
    x = "Cylinders", y = "Mean MPG") +
  theme_bw()
```



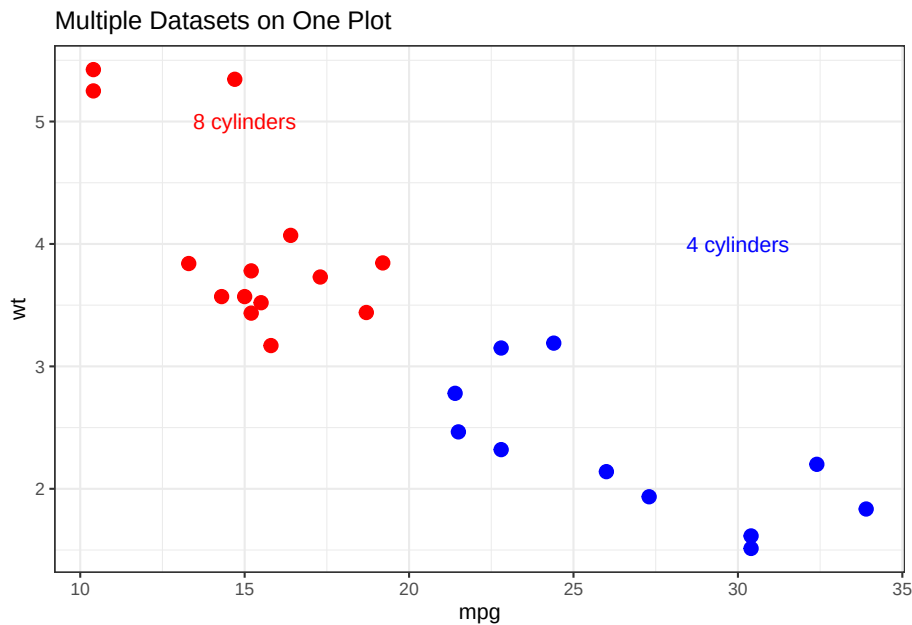
```
# Ribbon for confidence intervals
ggplot(data = mtcars, aes(x = mpg, y = wt)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm") +
  labs(title = "Linear Fit with Confidence Ribbon") +
  theme_bw()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
# Create two datasets
df1 <- mtcars[mtcars$cyl == 4, ]
df2 <- mtcars[mtcars$cyl == 8, ]

# Layer different datasets
ggplot() +
  geom_point(data = df1, aes(x = mpg, y = wt), color = "blue", size = 3) +
  geom_point(data = df2, aes(x = mpg, y = wt), color = "red", size = 3) +
  annotate("text", x = 30, y = 4, label = "4 cylinders", color = "blue") +
  annotate("text", x = 15, y = 5, label = "8 cylinders", color = "red") +
  labs(title = "Multiple Datasets on One Plot") +
  theme_bw()
```



4.15 Best Practices

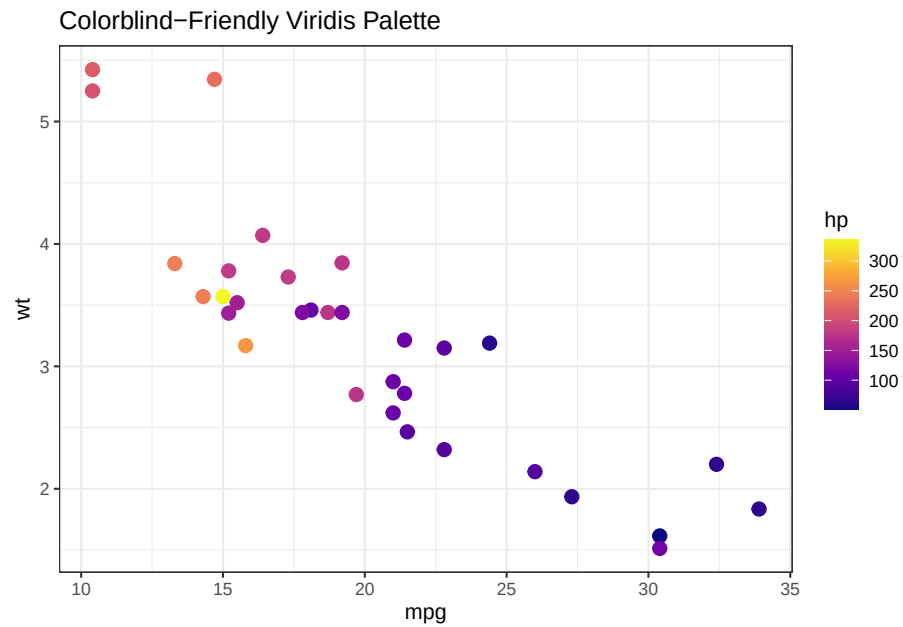
4.15.1 Choosing Appropriate Plot Types

- **Scatter plots:** Relationships between two continuous variables
- **Line plots:** Time series or sequential data
- **Bar plots:** Comparing categories or counts
- **Box plots:** Distribution comparisons across groups
- **Histograms/Density:** Distribution of a single variable
- **Heatmaps:** Matrices or two-way relationships

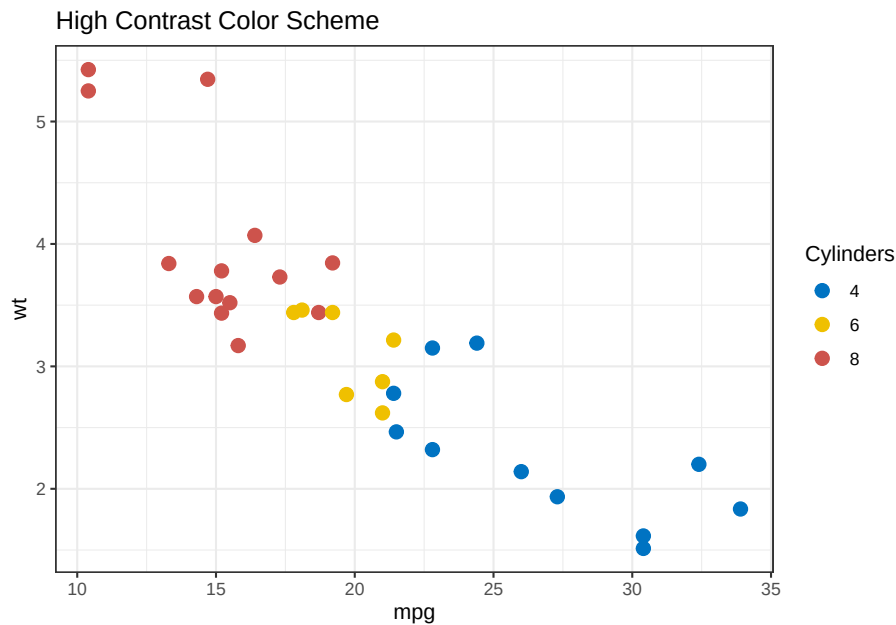
4.15.2 Color Accessibility

```
# Using colorblind-friendly palettes
library(viridis)

ggplot(data = mtcars, aes(x = mpg, y = wt, color = hp)) +
  geom_point(size = 3) +
  scale_color_viridis_c(option = "plasma") +
  labs(title = "Colorblind-Friendly Viridis Palette") +
  theme_bw()
```



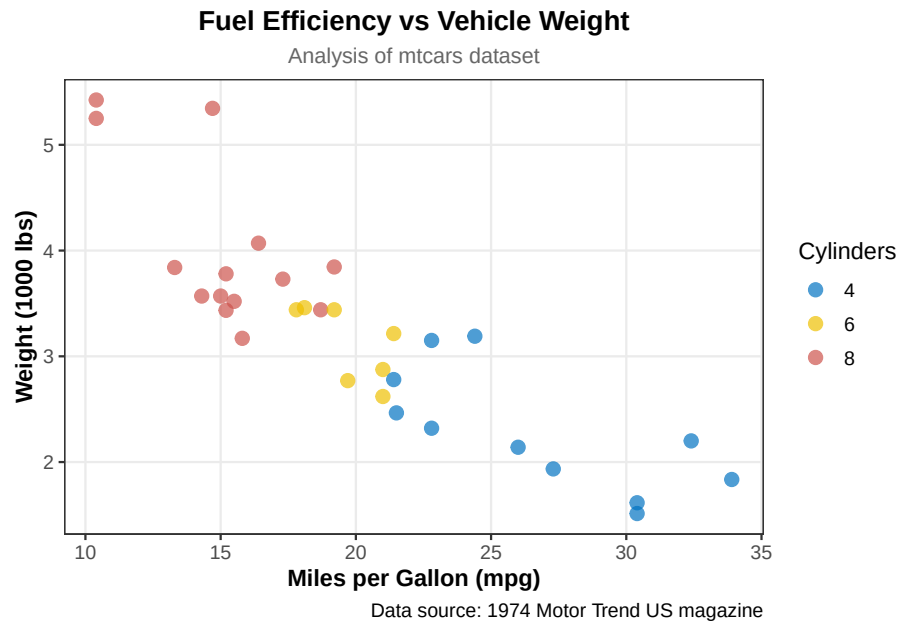
```
# High contrast for categories
ggplot(data = mtcars, aes(x = mpg, y = wt, color = factor(cyl))) +
  geom_point(size = 3) +
  scale_color_manual(values = c("4" = "#0073C2", "6" = "#EFC000", "8" = "#CD534C")) +
  labs(title = "High Contrast Color Scheme", color = "Cylinders") +
  theme_bw()
```



4.15.3 Publication-Quality Figures

```
# High-quality plot for publications
pub_plot <- ggplot(data = mtcars, aes(x = mpg, y = wt, color = factor(cyl))) +
  geom_point(size = 3, alpha = 0.7) +
  scale_color_manual(
    values = c("4" = "#0073C2", "6" = "#EFC000", "8" = "#CD534C"),
    name = "Cylinders"
  ) +
  labs(
    title = "Fuel Efficiency vs Vehicle Weight",
    subtitle = "Analysis of mtcars dataset",
    x = "Miles per Gallon (mpg)",
    y = "Weight (1000 lbs)",
    caption = "Data source: 1974 Motor Trend US magazine"
  ) +
  theme_bw(base_size = 12) +
  theme(
    plot.title = element_text(size = 14, face = "bold", hjust = 0.5),
    plot.subtitle = element_text(size = 11, hjust = 0.5, color = "gray40"),
    legend.position = "right",
    panel.grid.minor = element_blank(),
    axis.title = element_text(face = "bold")
  )
```

```
pub_plot
```



4.16 Saving Plots

You can save your plots to various file formats using the `ggsave()` function. Here's an example:

```
# Save as PDF (vector format - best for publications)
ggsave("scatter_plot.pdf", plot = pub_plot, width = 8, height = 6, dpi = 300)

# Save as PNG (raster format)
ggsave("scatter_plot.png", plot = pub_plot, width = 8, height = 6, dpi = 300)

# Save as SVG (vector format - good for web)
ggsave("scatter_plot.svg", plot = pub_plot, width = 8, height = 6)

# Save with specific dimensions
ggsave("scatter_plot_large.pdf", plot = pub_plot,
      width = 10, height = 8, units = "in", dpi = 300)
```

Note: Vector graphics formats like PDF and SVG are preferred for saving plots as they maintain quality when resized. Raster formats like PNG and JPEG may lose quality when resized. Always use `dpi = 300` or higher for publication-quality raster images.

Chapter 5

Statistical Analysis

R is the most widely used statistical programming language. It has functions for a wide variety of statistical analyses and is the go to language for statistical analysis.

5.1 Descriptive Statistics

```
mpg <- mtcars$mpg    # Extract the mpg column from the mtcars data set  
mean(mpg)           # Mean
```

```
## [1] 20.09062
```

```
median(mpg) # Median
```

```
## [1] 19.2
```

```
sd(mpg)       # Standard deviation
```

```
## [1] 6.026948
```

```
var(mpg)      # Variance
```

```
## [1] 36.3241
```

```
range(mpg)    # Range
```

```
## [1] 10.4 33.9
```

```
min(mpg)      # Minimum
```

```
## [1] 10.4
```

```
max(mpg)      # Maximum
```

```
## [1] 33.9
```

```
quantile(mpg) # Quantiles
```

```
##      0%    25%    50%    75%   100%
## 10.400 15.425 19.200 22.800 33.900
```

```
summary(mpg) # Summary
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  10.40   15.43   19.20   20.09   22.80   33.90
```

All of these functions come from the base R package and are available by default. However, there are many other packages that provide additional functions for statistical analysis. The most popular of these is the `dplyr` package.

```
# install.packages("dplyr") # Install the dplyr package
library(dplyr) # Load the dplyr package
```

```
mpg <- mtcars$mpg # Extract the mpg column from the mtcars data set
mean(mpg) # Mean
```

```
## [1] 20.09062
```

```
median(mpg) # Median
```

```
## [1] 19.2
```

You can also use packages like `psych` and `Hmisc` for more advanced descriptive statistics. Below is show how to use the `psych` package.

```
# install.packages("psych") # Install the psych package
library(psych) # Load the psych package
```

```
mpg <- mtcars$mpg # Extract the mpg column from the mtcars data set
describe(mpg) # Descriptive statistics
```

```
##      vars  n mean  sd median trimmed  mad  min  max range skew kurtosis  se
## X1      1 32 20.09 6.03   19.2    19.7 5.41 10.4 33.9  23.5 0.61    -0.37 1.07
```

5.2 t-Test

T-tests are used to compare the means of two groups. Statistics is out of the scope of this document, I recommend you to read more about t-tests in R from *Modern Statistics with R*. We will perform a t-test to compare the mpg of automatic and manual cars in the `mtcars` data set.

```
mpg_auto <- mtcars$mpg[mtcars$am == 0] # Extract the mpg of automatic cars
mpg_manual <- mtcars$mpg[mtcars$am == 1] # Extract the mpg of manual cars
t.test(mpg_auto, mpg_manual) # t-test
```

```
##
## Welch Two Sample t-test
##
## data: mpg_auto and mpg_manual
## t = -3.7671, df = 18.332, p-value = 0.001374
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## -11.280194 -3.209684
## sample estimates:
## mean of x mean of y
## 17.14737 24.39231
```

5.3 Correlation

```
mpg <- mtcars$mpg # Extract the mpg column from the mtcars data set
wt <- mtcars$wt # Extract the wt column from the mtcars data set
cor(mpg, wt) # Correlation
```

```
## [1] -0.8676594
```

```
cor.test(mpg, wt) # Correlation test
```

```
##
## Pearson's product-moment correlation
##
## data: mpg and wt
## t = -9.559, df = 30, p-value = 1.294e-10
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## -0.9338264 -0.7440872
## sample estimates:
## cor
## -0.8676594
```